UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR
DE INGENIEROS DE TELECOMUNICACIÓN

MÁSTER UNIVERSITARIO EN INGENIERÍA DE TELECOMUNICACIÓN

TRABAJO FIN DE MÁSTER

AN AUTOMATED FRAMEWORK FOR DATASET QUALITY ASSESSMENT AND DATA LEAKAGE DETECTION IN MACHINE LEARNING

JAIME CANO MORAÑO

2025–26

MÁSTER UNIVERSITARIO EN INGENIERÍA DE TELECOMUNICACIÓN

TRABAJO FIN DE MÁSTER

Título: An automated framework for dataset quality assessment and data leakage detection in machine learning

Autor: D. Jaime Cano Moraño

Tutor: D. Yong Zheng

Ponente: D. Pedro Reviriego Vasallo

Departamento: Departamento de Ingeniería de Sistemas Telemáticos

MIEMBROS DEL TRIBUNAL

Presidente: D. . . . . .

Vocal: D. . . . . .

Secretario: D. . . . . .

Suplente: D. . . . . .

Los miembros del tribunal arriba nombrados acuerdan otorgar la calificación de: . . . . . .

Madrid, a de de 20…

UNIVERSIDAD POLITÉCNICA DE MADRID

ESCUELA TÉCNICA SUPERIOR
DE INGENIEROS DE TELECOMUNICACIÓN

MÁSTER UNIVERSITARIO EN INGENIERÍA DE TELECOMUNICACIÓN

TRABAJO FIN DE MÁSTER

AN AUTOMATED FRAMEWORK FOR DATASET QUALITY ASSESSMENT AND DATA LEAKAGE DETECTION IN MACHINE LEARNING

JAIME CANO MORAÑO

2025–26

RESUMEN

El aprendizaje automático se ha convertido en una tecnología central en numerosos dominios de alto impacto, desde la sanidad hasta las finanzas o la justicia predictiva. Sin embargo, la fiabilidad de los modelos entrenados depende de manera crítica de la calidad de los datos utilizados: "*garbage in, garbage out*". Dos problemas especialmente dañinos son los defectos de calidad de los datos — como valores faltantes, duplicados, desequilibrio de clases o características de baja varianza — y la fuga de datos (*data leakage*), que ocurre cuando información que no estaría disponible en el momento de la predicción se filtra de forma inadvertida en el proceso de entrenamiento, produciendo modelos que parecen excelentes en validación pero fallan en producción.

Este Trabajo Fin de Máster presenta **ml-framework**, un sistema de código abierto, desarrollado en 17 fases incrementales, para la evaluación automática de la calidad de conjuntos de datos y la detección de fuga de datos en flujos de trabajo de aprendizaje automático. El framework implementa un *pipeline* de 22 comprobaciones organizadas en seis dimensiones — calidad, fuga, análisis de características, suficiencia estadística, detección de deriva e impacto sobre el rendimiento — y es accesible mediante tres interfaces complementarias: una herramienta de línea de comandos, un SDK de Python (`DatasetChecker`) y una aplicación web interactiva basada en Streamlit. Un sistema de *plugins* (`@register_check`) permite a terceros añadir comprobaciones personalizadas sin modificar el núcleo del framework.

La contribución más novedosa es la *puntuación unificada de riesgo de fuga*, $\mathcal{L}(f) = 0.35\,\rho(f) + 0.35\,\tilde{I}(f;y) + 0.30\,\pi(f)$, que combina correlación de Pearson/Cramér, información mutua normalizada e inflación del rendimiento en una sola puntuación interpretable. Complementariamente, un módulo basado en GPT-4o-mini (Azure OpenAI) detecta fuga semántica — casos en los que el nombre de una característica revela información futura o *post hoc* que los métodos estadísticos no pueden capturar, como una columna `boat` en el célebre conjunto de datos del Titanic.

La evaluación, respaldada por una batería de 325 pruebas automatizadas, se realiza sobre **doce conjuntos de datos** (seis sintéticos diseñados para aislar problemas específicos y seis del repositorio UCI: Titanic, Pima Diabetes, Adult Census Income, German Credit, Heart Disease y Wine Quality) y demuestra una tasa de detección de 4/4 en escenarios de fuga construidos, puntuaciones de disponibilidad calibradas de 71.6 (B, `leaky_dataset`) a 98.8 (A, `clean_dataset`), y una cobertura de $2.6\times$ más elementos de la lista de comprobación de comparación (29/29) que la mejor alternativa existente (Deepchecks, ydata-profiling, Great Expectations). El módulo semántico alcanza, sobre un simulador determinista que valida el arnés de evaluación (no una llamada real a GPT-4o-mini), una precisión de 1.00 y un *recall* de 0.93 (F1 = 0.96) sobre un benchmark de 30 características etiquetadas manualmente. Tres casos de estudio reales (Titanic, Adult Census e German Credit) ilustran el uso del framework de extremo a extremo. Todos los pesos y parámetros del sistema son configurables mediante un fichero YAML para facilitar la reproducibilidad y el análisis de sensibilidad.

Palabras clave: calidad de datos, fuga de datos, aprendizaje automático centrado en datos, puntuación de disponibilidad, información mutua, modelos de lenguaje, GPT-4o-mini.

SUMMARY

Machine learning has become a core technology across high-impact domains ranging from healthcare to finance and predictive justice. Yet the reliability of trained models depends critically on the quality of the data used — "*garbage in, garbage out*". Two particularly damaging problems are dataset quality defects—missing values, duplicates, class imbalance, low-variance features—and *data leakage*, where information unavailable at prediction time inadvertently enters the training process, producing models that look excellent during validation yet fail in production.

This thesis presents **ml-framework**, an open-source system, developed over 17 incremental phases, for the automated assessment of dataset quality and the detection of data leakage in machine learning workflows. The framework implements a 20-check pipeline organised across five dimensions — quality, leakage, feature analysis, statistical sufficiency, and drift detection — plus an impact-analysis stage that quantifies performance impact, and is exposed through three complementary interfaces: a command-line tool, a Python SDK (`DatasetChecker`), and an interactive Streamlit web application. A plugin system (`@register_check`) allows third parties to add custom checks without modifying the framework core.

The core algorithmic contribution is the *unified leakage risk score* $\mathcal{L}(f) = 0.35\,\rho(f) + 0.35\,\tilde{I}(f;y) + 0.30\,\pi(f)$, which combines Pearson or Cramér's correlation, normalised mutual information, and per-feature performance inflation into a single interpretable score. An optional module based on GPT-4o-mini (Azure OpenAI) extends this with *semantic leakage* detection, identifying features whose names reveal post-hoc or temporal information that purely statistical methods cannot detect—such as a `boat` column in the well-known Titanic dataset.

Backed by a suite of 325 automated tests, the evaluation spans **twelve datasets** (six synthetic datasets each designed to isolate a specific issue, and six real-world UCI datasets: Titanic, Pima Diabetes, Adult Census Income, German Credit, Heart Disease, and Wine Quality). Results demonstrate 4/4 leakage detection on constructed scenarios, calibrated readiness scores ranging from 71.6 (grade B, `leaky_dataset`) to 98.8 (grade A, `clean_dataset`), and $2.6\times$ more items on a 29-item cross-tool capability checklist than the best competing tool (Deepchecks, ydata-profiling, Great Expectations). On a deterministic offline simulation that validates the evaluation harness (not a live GPT-4o-mini call), the semantic module achieves precision 1.00 and recall 0.93 (F1 = 0.96) on a manually labelled 30-feature benchmark; a live-model evaluation is left as future work. Three real-world case studies (Titanic, Adult Census, and German Credit) illustrate the end-to-end use of the framework. All framework parameters are configurable via a YAML file to support reproducibility and sensitivity analysis.

The framework is fully open-sourced at `https://github.com/jaimecano12/ml-framework`.

Keywords: data quality, data leakage, data-centric AI, readiness score, mutual information, large language models, GPT-4o-mini.

ACRONYMS

API	Application Programming Interface
CD	Direct Costs (*Costes Directos*, Annex B)
CI	Indirect Costs (*Costes Indirectos*, Annex B)
CLI	Command-Line Interface
CV	Cross-Validation
GPT	Generative Pre-trained Transformer
HTML	HyperText Markup Language
IQR	Interquartile Range
JSON	JavaScript Object Notation
KS	Kolmogorov–Smirnov (statistical test)
LLM	Large Language Model
LRS	Leakage Risk Score
MI	Mutual Information
ML	Machine Learning
PSI	Population Stability Index
SDK	Software Development Kit
TFM	Trabajo Fin de Máster (Master's Thesis)
UCI	University of California, Irvine (Machine Learning Repository)
UPM	Universidad Politécnica de Madrid
VAT	Value Added Tax (*IVA*, Annex B)
YAML	YAML Ain't Markup Language

Contents

List of Figures

List of Tables

1. Introduction and Objectives

1.1 Introduction

In recent years, the rapid development of machine learning techniques has enabled the creation of increasingly powerful predictive models applied across many domains, including finance, healthcare, natural language processing, and autonomous systems. However, the performance and reliability of these models depend critically on the quality of the datasets used during training. Ensuring dataset quality and detecting potential issues before model training has therefore become a central task in modern machine learning workflows.

This chapter introduces the theoretical background required to understand the context and motivation of this project. It presents an overview of the current machine learning landscape, discusses common problems related to dataset quality and data leakage, reviews existing validation tools, and explains the motivation for developing the proposed framework.

1.1.1 Machine Learning Context

Machine learning (ML) allows computer systems to learn patterns from data and make predictions or decisions without being explicitly programmed for each task. ML models are now widely deployed in healthcare diagnosis, credit scoring, recommendation systems, industrial predictive maintenance, and many other areas where reliable, data-driven decisions matter.

Traditionally, progress in ML has been associated with improvements in algorithms and architectures. However, recent research has highlighted that the quality of the data used to train these models plays an equally important—and often more decisive—role. This shift in perspective is sometimes described as the *data-centric AI* paradigm, where the focus moves from model architecture to the systematic improvement of datasets and data pipelines [16].

In practice, ML systems rely on datasets that may be collected from heterogeneous sources under imperfect conditions. If the data is incomplete, biased, inconsistent, or contains hidden correlations with the target variable, the resulting models may produce unreliable predictions or overly optimistic evaluation results. Dataset validation and quality assessment have therefore become essential steps in the development of trustworthy ML systems.

1.1.2 Dataset Quality Challenges

Several categories of data quality problems commonly arise in machine learning datasets:

- **Missing values.** Data collected from heterogeneous sources often contains incomplete records due to measurement failures, data entry errors, or optional survey questions. Missing values can degrade model accuracy and introduce bias if not handled appropriately [10].
- **Duplicate rows.** Exact or near-duplicate observations may result from data integration errors. Duplicates that span the training and test split create inflated evaluation metrics by allowing the model to memorise rather than generalise.

- **Outliers.** Anomalous values may distort statistical properties and impair certain learning algorithms, particularly those sensitive to the scale or distribution of inputs.
- **Class imbalance.** When one class is strongly underrepresented, models tend to become biased toward the majority class, producing misleading aggregate accuracy while failing on minority cases—a critical problem in fraud detection, medical diagnosis, and anomaly identification.
- **Constant and low-variance features.** Features that carry little or no variation across samples provide no discriminative signal and may slow convergence or introduce numerical instability.
- **Highly correlated features.** Strongly collinear features add redundancy, complicate coefficient interpretation, and may hurt regularised models.

Dataset quality issues should not be understood merely as data engineering problems: they are factors that directly affect ML model behaviour. Poor-quality data can reduce predictive accuracy, increase variance, generate unstable predictions, and amplify bias. In many practical scenarios, improving dataset quality produces larger performance gains than switching model architectures [13].

1.1.3 Data Leakage in Machine Learning

Data leakage is one of the most critical and often underestimated problems in machine learning workflows. Leakage occurs when information that would not normally be available at prediction time is inadvertently used during training or evaluation. The result is a model that appears to perform very well on held-out data while failing in production, because it has learned to exploit illegitimate signals rather than genuine predictive patterns.

Leakage takes several forms:

Target leakage. A feature is directly or indirectly derived from the target variable, encoding outcome information that would not be available at inference time. A classic example is including a hospital patient's `discharge_code` as a feature when predicting readmission: the code is assigned only after the patient leaves, making it a perfect—but illegitimate—predictor. Kaufman et al. [1] provide a taxonomy of leakage types and demonstrate how undetected leakage leads to inflated and misleading performance estimates in published studies.

Train-test contamination. Preprocessing operations such as normalisation, feature scaling, or imputation applied to the full dataset before splitting contaminate the test set with information from training samples. Even if no feature is individually leaky, this procedural error can inflate metrics.

Temporal leakage. In time-ordered datasets, using future observations to predict past events violates the chronological structure of the problem. This is especially common in finance, demand forecasting, and healthcare analytics. A dataset that is not sorted chronologically will produce an unrealistic train-test split if divided naively.

Identifier leakage. High-cardinality columns such as patient IDs, order codes, or UUIDs are often inadvertently left in the feature set. A model can memorise these identifiers during training, achieving near-perfect accuracy on the training set and high accuracy on overlapping test observations while generalising poorly.

Semantic leakage. Even when a feature passes all statistical checks, its name may reveal that it encodes future or post-hoc information. A feature named `future_usage_days` or `salary_after_promotion` conveys temporal information that no correlation measure can detect.

Leakage is particularly dangerous because it is often invisible during development. A model trained on leaky data may achieve near-perfect validation metrics, creating a false sense of confidence that persists until the model is deployed and tested against genuinely unseen data.

1.1.4 Existing Dataset Validation Approaches

Several tools address aspects of dataset validation for machine learning:

Great Expectations [18] is an open-source library that enables users to define data expectations (constraints) and validate datasets against them. It supports pipeline integration and generates HTML reports, but its focus is on general data contract enforcement rather than ML-specific leakage detection. Users must manually author each expectation; the tool does not automatically discover leakage risk or compute a readiness score.

ydata-profiling [17] generates comprehensive HTML exploratory reports covering missing values, distributions, correlations, and duplicate rows. It is extremely useful for interactive EDA but does not provide actionable ML-specific scores, leakage detection, or code-level recommendations.

Deepchecks [19] offers a rich suite of checks for both datasets and models, including train-test distribution comparisons. It provides model-level integrity checks that complement dataset validation but does not implement a unified leakage risk score or support the full range of sufficiency and drift checks.

A particularly relevant contribution from industry is **TensorFlow Data Validation (TFDV)**, developed by Google as part of the TensorFlow Extended (TFX) ecosystem. TFDV computes descriptive statistics for datasets, detects anomalies against a learned schema, and monitors distribution shift between training and serving data. It integrates seamlessly into TFX production pipelines and scales to very large datasets. However, it requires TensorFlow infrastructure, focuses on schema validation and distribution monitoring rather than ML-specific leakage detection, and does not provide a composite readiness score or code-level corrective recommendations.

The **ML Test Score** rubric proposed by Breck et al. [3] defines a conceptual framework of 28 tests for evaluating the production-readiness of ML systems. Among its data-related tests, it covers feature monitoring, distribution validation, and documentation of expected data characteristics. This rubric is not an automated tool, but it provides a valuable conceptual basis for the checks implemented in ml-framework.

Despite their strengths, all of these tools share a common limitation: none provides a *unified leakage risk score* that combines multiple statistical signals, and none integrates LLM-based

semantic analysis to catch implicit leakage from feature naming conventions. Table 1.1 summarises the comparison.

Table 1.1. Comparison of dataset validation approaches.

Tool	Main focus	Leakage	ML impact	Score	Recs.
Great Expectations	Data contracts / rules	No	No	No	No
ydata-profiling	EDA / data profiling	No	No	No	No
Deepchecks	Dataset + model integrity	No	Partial	No	No
TFDV	Statistical validation	No	No	No	No
ml-framework	Quality + leakage + drift	**Yes**	**Yes**	**Yes**	**Yes**

1.1.5 Motivation of the Project

The gap described above motivates ml-framework. The practical problem is straightforward: data scientists and ML engineers routinely spend significant time debugging models that underperform in production because of data issues that could have been detected automatically before training. Leakage is especially costly because it is invisible during development and manifests only after deployment.

The technical challenge is that no single check is sufficient, and three types of evidence support a multi-signal approach. First, linear correlation catches obvious leakage but is insufficient on its own: a feature with a nonlinear monotone relationship to the target may have low Pearson $|r|$ while remaining highly informative. Second, mutual information captures these nonlinear associations but can be noisy for small samples and high-cardinality features. Third, performance inflation provides the most direct evidence—retraining a model with and without a feature—but is computationally expensive when applied to every feature and sensitive to the chosen learner. Combining all three signals in a principled, weighted score compensates for the weaknesses of each individual method.

The semantic leakage problem adds a further dimension entirely outside the scope of statistical methods: a practitioner reviewing feature names may immediately recognise that `discharge_code` or `boat` is a post-hoc variable, but no correlation measure can detect this from the data values alone. Instruction-following language models capable of domain reasoning provide a natural solution for this class of leakage, motivating the GPT-4o-mini semantic module described in Chapter 2.

Finally, the fragmentation of existing tools—each covering a different subset of checks with a different interface, as summarised in Table 1.1—makes systematic validation difficult. A single YAML-driven pipeline with a composite readiness score eliminates this fragmentation and enables direct comparison across datasets, teams, and time.

ml-framework addresses this gap by providing:

1. A complete, automated pipeline covering quality, leakage, feature analysis, sufficiency, drift, and performance impact in a single configurable tool, exposed via CLI, Python SDK, and an interactive Streamlit dashboard.

2. A novel unified leakage risk score combining correlation, mutual information, and performance inflation into a single interpretable, configurable metric.

3. An optional GPT-4o-mini semantic module for detecting implicit leakage from feature names and domain context, validated on a manually labelled benchmark.

4. A composite readiness score (0–100, grade A–F) with code-level recommendations, evaluated end-to-end on twelve datasets and benchmarked against three established tools.

1.2 Objectives

The main objective of this project is to design, implement, and evaluate an automated framework for dataset quality assessment and data leakage detection in machine learning workflows.

The specific objectives are:

1. Implement a comprehensive pipeline of 20 automated checks covering quality, leakage, feature analysis, sufficiency, and drift, plus an impact-analysis stage.
2. Develop a novel unified leakage risk score that combines three complementary signal types (correlation, mutual information, performance inflation) into an interpretable per-feature score.
3. Integrate an LLM-based semantic leakage module capable of detecting implicit leakage from feature names and dataset descriptions.
4. Construct a quantitative evaluation benchmark and compare the framework against three existing tools (Great Expectations, ydata-profiling, Deepchecks).
5. Provide multiple user interfaces: a CLI, a Python SDK, and an interactive Streamlit web application.
6. Make all framework parameters configurable via a YAML configuration file to ensure reproducibility and support sensitivity analysis.
7. Evaluate the framework on twelve real-world and synthetic datasets representing different defect profiles.

2. Development

This chapter describes the full development of **ml-framework**: the state of the art against which it is positioned (Section 2.1), its architecture and user-facing design (Section 2.2), the statistical and algorithmic methodology underlying every check (Section 2.3), the engineering choices behind its implementation (Section 2.4), and finally a chronological overview of the seventeen incremental phases through which the system was built (Section 2.5).

2.1 State of the Art and Related Work

This section situates ml-framework with respect to prior academic work and existing open-source tools. Section 1.1.4 of Chapter 1 already introduced the four tools compared throughout this thesis (Great Expectations, ydata-profiling, Deepchecks, and TFDV) and Table 1.1; the subsections below review the broader academic literature underpinning each component of the framework.

2.1.1 Data Leakage Detection

The problem of data leakage in machine learning has been studied from multiple angles. Kaufman et al. [1] provide the most comprehensive taxonomy, identifying target leakage, train-test contamination, and temporal leakage as the three primary subtypes. They show that published ML studies in finance and medicine frequently contain undetected leakage, leading to inflated performance claims that do not replicate in practice. Sculley et al. [2] describe the broader class of hidden technical debt in ML systems, of which leakage is a particularly severe instance because it is self- concealing: a model trained on leaked features looks better, not worse, than one trained on clean data.

More recent work has focused on detecting specific leakage patterns. Narayan et al. [14] propose DoubleML, a causal inference framework that can detect features whose contribution to model performance exceeds what their direct correlation with the target can explain—a signal similar to the performance inflation component in our unified risk score. Sui et al. [15] investigate leakage detection in automated ML pipelines, showing that standard AutoML tools are particularly vulnerable to preprocessor-induced contamination.

2.1.2 Dataset Quality Assessment

Breck et al. [3] define a production readiness rubric for ML systems with 28 tests covering data, models, and infrastructure. Their data tests map closely to the quality checks in ml-framework (missing values, schema consistency, class balance) but do not include a unified leakage risk score or composite readiness score.

The data-centric AI movement, popularised by Ng [16], advocates systematic improvement of training data as a primary lever for ML performance. Zha et al. [13] formalise this through the concept of *data-centric benchmarks*, which evaluate how much model improvement is achievable through data cleaning alone—a perspective that motivates the impact analysis component of ml-framework.

2.1.3 Mutual Information and Feature Relevance

Mutual information (MI) is a non-parametric association measure from information theory that captures arbitrary dependence, including nonlinear relationships that Pearson correlation misses. Ross [7] and Kraskov et al. [6] develop estimators for continuous MI that are used in sklearn's `mutual_info_classif`—the estimator used in the unified leakage risk score. Normalising MI by its maximum observed value across all features provides a scale-invariant signal suitable for aggregation.

2.1.4 Distribution Shift and Drift Detection

Rabanser et al. [8] provide a comprehensive survey of data drift detection methods, covering two-sample tests (Kolmogorov-Smirnov, MMD), dimensionality-reduction approaches, and drift-specific metrics such as Population Stability Index (PSI). ml-framework implements KS test plus PSI for covariate drift detection and applies Bonferroni correction for multiple-feature comparisons, consistent with the recommendations in their survey.

2.1.5 LLMs for Data Analysis

OpenAI's GPT-4 family [11] has demonstrated strong performance on information extraction and reasoning tasks involving structured data descriptions. Narayan et al. [14] show that foundation models can perform data wrangling tasks—including type inference, entity resolution, and constraint checking—with competitive accuracy relative to specialised tools. Sui et al. [15] demonstrate that LLMs can synthesise entire data science pipelines from natural language descriptions. Several recent papers also use LLMs to generate data cleaning code, impute missing values, and detect data quality issues through natural language understanding.

Our semantic leakage module is, to our knowledge, the first application of an LLM specifically to the problem of detecting *semantic* leakage from feature names and dataset context, where the detection signal is entirely linguistic rather than statistical. The key insight is that GPT-4o-mini has been trained on large corpora that include descriptions of medical, financial, and operational datasets, giving it sufficient domain knowledge to recognise that a feature named `discharge_code` is likely post-hoc and that `future_usage` encodes future information.

2.1.6 Data-Centric AI

The data-centric AI paradigm, popularised by Ng [16], reorients the ML development process from model-centric ("find the best architecture") to data-centric ("systematically improve the training data"). Zha et al. [13] formalise this through data-centric benchmarks that measure how much model improvement is achievable through data cleaning and augmentation alone, finding that data quality improvements often outperform architecture changes. ml-framework operationalises this paradigm by providing actionable, code-level recommendations that can be directly applied to improve dataset quality before training begins.

2.2 System Architecture and Design

2.2.1 Design Principles

ml-framework is designed around three principles. **Comprehensiveness**: a single pipeline covers all major pre-training data quality dimensions. **Transparency**: every check produces an interpretable message with affected columns and a severity level; the composite readiness score is decomposed by dimension. **Configurability**: all parameters—thresholds, weights, penalties—are exposed in a single YAML file.

2.2.2 Pipeline Architecture

Figure 2.1 shows the end-to-end pipeline. A YAML configuration file and an input dataset are the only required inputs. The pipeline sequentially executes quality, leakage, feature analysis, sufficiency, drift, and impact modules, then aggregates results into recommendations and a composite readiness score before generating reports.

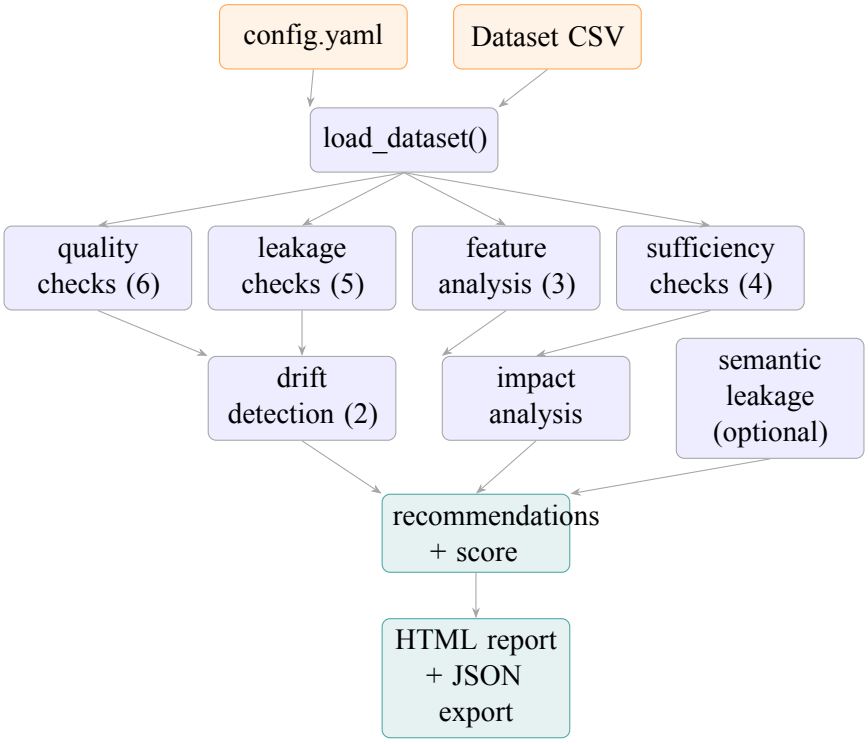

Figure 2.1. ml-framework pipeline. Arrows show data flow from configuration and dataset through six parallel analysis modules to recommendations, readiness score, and reports.

2.2.3 User Interfaces

The framework exposes three complementary interfaces:

Command-line interface (CLI). The `main.py` entry point accepts `--config`, `--dataset`, `--output-dir`, and `--log-level` flags. It runs the full pipeline and writes the HTML report to the specified output directory.

Python SDK. The `DatasetChecker` class provides a programmatic API for integration into Jupyter notebooks, CI/CD pipelines, and other Python tools.

Listing 2.1. Python SDK usage example.

```python
from src.checker import DatasetChecker

checker = DatasetChecker("configs/config.yaml")
report  = checker.run("data/titanic.csv", target_col="survived")

print(f"Score: {checker.score}/100  Grade: {checker.grade}")
for rec in checker.top_recommendations(priority="high"):
    print(f"  [{rec.check_name}] {rec.action}")

checker.save_report("reports/")
```

Streamlit web application. `app/app.py` provides a seven-tab interactive dashboard: dataset overview, quality checks, leakage detection, feature analysis, sufficiency, impact analysis, and a readiness score summary with downloadable report.

2.2.4 Module Structure

Table 2.1 lists the fourteen source modules and their responsibilities.

Table 2.1. Source module summary.

Module	Phase	Responsibility
`utils.py`	1	Shared dataclasses: CheckResult, FrameworkReport, ReadinessScore
`config.py`	2	YAML loading, deep-merge, section validation
`quality_checks.py`	3	Six quality checks (missing, duplicates, outliers, imbalance, constant, low-variance)
`leakage_checks.py`	4,16	Five leakage checks including unified risk score
`impact_analysis.py`	5	Cross-validated baseline vs. cleaned accuracy delta
`reporting.py`	6	Jinja2 HTML report with embedded Matplotlib figures
`recommendations.py`	8	20 rule handlers producing code-level Recommendation objects
`feature_analysis.py`	9	Correlation, mutual information relevance, distribution shape
`scoring.py`	10	Composite readiness score (0–100) with configurable weights
`sufficiency.py`	11	Four statistical sufficiency checks
`checker.py`	13	DatasetChecker SDK orchestrating the full pipeline
`drift_checks.py`	14	KS test and PSI covariate/label drift
`plugins.py`	15	@register_check plugin API for custom checks
`semantic_leakage.py`	16	GPT-4o-mini semantic leakage module

2.2.5 Complete Check Catalog

Table 2.2 provides a complete reference of all 21 automated checks implemented in the framework, organised by module: 20 checks run deterministically in every pipeline execution (quality, leakage, features, sufficiency, drift), plus the optional `semantic_leakage` check, which requires Azure OpenAI credentials and is disabled by default. A separate impact-analysis stage contributes one additional result per configured model (one to three, depending on configuration) but is not listed in this table, as it evaluates predictive performance rather than a single dataset property.

Table 2.2. Complete catalog of ml-framework checks (21 total: 20 deterministic + 1 optional semantic check).

Module	Check name	Method	Default threshold
Quality	missing_values	NaN rate	$> 5\%$ per column
Quality	duplicates	Exact row match	Any duplicate
Quality	outliers	IQR (k=3.0) or z-score	$> 3\sigma$
Quality	class_imbalance	Minority class ratio	$< 10\%$
Quality	constant_features	`nunique`≤ 1	Any constant
Quality	low_variance	CV = std/|mean|	< 0.01
Leakage	target_leakage	Pearson $\|r\|$ / Cramér's V	≥ 0.95
Leakage	train_test_overlap	Duplicate rows in split	Any overlap
Leakage	temporal_leakage	Monotonic date ordering	Unsorted
Leakage	id_column_leakage	Unique-value ratio	$\geq 95\%$
Leakage	leakage_risk_score	$\mathcal{L}(f) = 0.35\rho + 0.35\tilde{I} + 0.30\pi$	≥ 0.7
Features	feature_correlation	Pearson $\|r\|$ between features	≥ 0.90
Features	feature_relevance	Normalised MI with target	< 0.01
Features	distribution_shape	Skewness / kurtosis	|skew|>2, kurtosis>7
Sufficiency	sample_size	Absolute row count	< 100
Sufficiency	class_support	Samples per class	< 30
Sufficiency	cv_stability	Std of 5-fold CV accuracy	> 0.10
Sufficiency	feature_sample_ratio	p/n	> 0.10
Drift	covariate_drift	KS test + Bonferroni + PSI	$p < 0.05$ / PSI> 0.2
Drift	label_drift	Distribution shift in y	$p < 0.05$
Semantic	semantic_leakage	GPT-4o-mini LLM analysis	risk $\geq$ medium

2.2.6 Configuration System

All framework parameters are stored in a YAML configuration file. The file is structured into top-level blocks corresponding to each module: `quality_checks`, `leakage_checks`, `feature_analysis`, `sufficiency_checks`, `drift_checks`, `impact_analysis`, `scoring`, and `semantic_leakage`. A representative excerpt of the configuration file is shown in Listing 2.2.

Listing 2.2. Excerpt from configs/config.yaml showing configurable parameters.

```yaml
leakage_checks:
  target_leakage:
    correlation_threshold: 0.95
  leakage_risk_score:
    threshold: 0.7
    weights: [0.35, 0.35, 0.30]    # correlation, MI, performance
    cv_folds: 3

scoring:
  dimension_weights:
    quality:     0.25
```

```
    leakage:      0.35
    features:     0.25
    sufficiency:  0.15
  severity_penalties:
    error:    15.0
    warning:   5.0
  leakage_multiplier: 1.5
  grade_thresholds:
    A: 85
    B: 70
    C: 55
    D: 40

semantic_leakage:
  enabled: false
  deployment: gpt-4o-mini
  risk_threshold: medium
```

Deep-merge semantics allow dataset-specific override files to inherit all defaults while overriding only the relevant keys. This design means that a sensitivity analysis can be run by programmatically sweeping a single parameter without modifying any source code:

Listing 2.3. Programmatic parameter sweep via the SDK.

```
for threshold in [0.85, 0.90, 0.95]:
    checker = DatasetChecker("configs/config.yaml")
    checker.set(leakage_checks__target_leakage__correlation_threshold=
        threshold)
    report = checker.run("data/titanic.csv", target_col="survived")
    print(f"threshold={threshold}  score={checker.score}")
```

2.3 Core Methodology

2.3.1 Quality Checks

Six quality checks are implemented, each returning a `CheckResult` with a severity level (`error`/`warning`/`info`), an explanatory message, affected columns, and structured details. Table 2.3 summarises the checks.

Table 2.3. Quality checks implemented in ml-framework.

Check	Method	Description
Missing values	NaN rate $>$ threshold	Flags columns exceeding configurable missing-value rate
Duplicates	Exact row match	Detects identical rows; computes overlap after simulated split
Outliers	IQR or z-score	Identifies values beyond $Q_3 + k \cdot$ IQR or k standard deviations
Class imbalance	Minority ratio	Flags targets where any class falls below a configurable threshold
Constant features	`nunique`≤ 1	Detects columns with zero variance
Low variance	CV $=$ std/$\lvert$mean$\rvert$	Flags numeric columns with coefficient of variation below threshold

2.3.2 Leakage Detection

Classical Checks

Four classical leakage checks address the most common leakage patterns encountered in practice.

Target leakage. For each feature, the framework computes a feature-target association score: Pearson $|r|$ for numeric features, Cramér's V for categorical features. Features exceeding a configurable threshold (default 0.95) are flagged as `error` severity.

Train-test overlap. The dataset is split by a configurable random seed and test ratio. After deduplication, the number of rows appearing in both halves is reported. Overlap is only possible when duplicate rows exist.

Temporal leakage. When a date column is configured, the check verifies that the dataset is sorted chronologically. An unsorted date column means a naïve sequential split would mix past and future observations.

ID column leakage. Columns where the unique-value ratio exceeds a configurable threshold (default 95%) are flagged. Float columns are excluded because high cardinality is expected for continuous measurements.

Unified Leakage Risk Score

The classical checks are heuristic: each produces a binary pass/fail verdict based on a single threshold. The unified leakage risk score combines three complementary signals into a continuous per-feature risk measure:

$$\mathcal{L}(f) = w_\rho \cdot \rho(f) + w_I \cdot \tilde{I}(f; y) + w_\pi \cdot \pi(f) \tag{2.1}$$

where:

- $\rho(f) \in [0, 1]$ is Pearson $|r|$ or Cramér's V, measuring linear or categorical association.
- $\tilde{I}(f; y) = I(f; y) / \max_j I(f_j; y) \in [0, 1]$ is mutual information normalised by the maximum MI across all features, measuring arbitrary (including nonlinear) dependence.
- $\pi(f) \in [0, 1]$ is the performance inflation: the fractional improvement in single-feature logistic regression accuracy over the majority-class baseline, normalised to $[0, 1]$.
- Default weights: w_ρ=0.35, w_I=0.35, w_π=0.30.

Features with $\mathcal{L}(f) \geq 0.7$ are flagged as `warning`; $\mathcal{L}(f) \geq 0.9$ as `error`. Algorithm 1 gives the full procedure.

2.3.3 Feature Analysis

Three feature analysis checks complement the leakage detection module. **Feature correlation** flags pairs of features whose absolute Pearson correlation exceeds a threshold (default 0.90), signalling redundancy. **Feature relevance** uses normalised mutual information to flag features with near-zero association with the target ($\tilde{I} < 0.01$ by default), which are candidates for removal. **Distribution shape** computes skewness and excess kurtosis; features with |skew| > 2

Algorithm 1 Unified Leakage Risk Score computation.

Require: DataFrame $\mathbf{X}$, target column y, config (weights, threshold, cv_folds)
Ensure: Per-feature risk scores $\{\mathcal{L}(f)\}$

for each feature $f \in \mathbf{X}$ **do**
  $\rho(f) \leftarrow$ FeatureTargetAssoc($\mathbf{X}[f]$, y) ▷ Pearson $|r|$ or Cramér's V
end for
$\mathbf{m} \leftarrow$ MutualInfoClassif($\mathbf{X}$, y) ▷ sklearn estimator, random_state=42
$\tilde{I}(f) \leftarrow \mathbf{m}[f] / \max(\mathbf{m})$ for each f
$b \leftarrow$ majority-class accuracy baseline
for each feature $f \in \mathbf{X}$ **do**
  $a_f \leftarrow$ CrossValScore(LogisticRegression, $\mathbf{X}[[f]]$, y, cv)
  $\pi(f) \leftarrow \max(0,\ a_f - b)/(1 - b)$
end for
$\mathcal{L}(f) \leftarrow \min(1,\ w_\rho\, \rho(f) + w_I\, \tilde{I}(f) + w_\pi\, \pi(f))$ for each f
return $\{\mathcal{L}(f)\}$

or excess kurtosis > 7 are flagged, as these can impair gradient-based learners and distance-based methods.

2.3.4 Statistical Sufficiency

Four sufficiency checks evaluate whether the dataset has enough samples to support reliable training: (1) a minimum absolute row count, (2) a minimum samples per class, (3) cross- validation stability (the standard deviation of 5-fold accuracy must not exceed a threshold), and (4) the feature-to-sample ratio p/n (flagged when $p/n > 0.10$, indicating risk of overfitting with unregularised models).

2.3.5 Drift Detection

Covariate drift between dataset halves (split by a date column, or by index when no date column is configured) is assessed using the two-sample Kolmogorov-Smirnov (KS) test with Bonferroni correction:

$$\text{reject } H_0^{(j)} \text{ if } p_j < \alpha/p \tag{2.2}$$

where $\alpha = 0.05$ and p is the number of features tested. Label drift (shift in the class distribution) is tested separately. Population Stability Index (PSI) is also computed and flagged when PSI > 0.2 (industry standard for significant shift).

2.3.6 Impact Analysis

The impact analysis module quantifies the practical consequence of a dataset defect by measuring the performance *delta* between a baseline model trained on the original data and the same model trained after the defect is removed. Three models are evaluated (logistic regression, random forest, XGBoost) under 5-fold cross-validation, and three metrics are reported (accuracy, ROC-AUC, F1). A positive Δ implies that the original data contained signal that is lost when

the defect is removed—evidence of leakage. A negative Δ implies that the defect was genuinely hurting model performance.

2.3.7 Readiness Score

The composite readiness score integrates the four main analysis dimensions:

$$S = 100 - \left(w_q \cdot P_q + w_l \cdot P_l + w_f \cdot P_f + w_s \cdot P_s\right) \tag{2.3}$$

where P_d is the total penalty for dimension d (errors -15 pts; warnings -5 pts), and the default weights are w_q=0.25, w_l=0.35, w_f=0.25, w_s=0.15. Leakage penalties are further amplified by a $1.5\times$ multiplier.

Weight rationale. Leakage receives the highest weight because it is a silent failure mode: a leaky model appears to perform well during validation while generalising poorly in production [1]. Quality and feature issues are equally weighted because both impair learning but are typically correctable. Sufficiency contributes least because sample-size adequacy depends strongly on the learner. Penalties were calibrated so that a single confirmed leaky feature is sufficient to drop a dataset from grade A to grade B. All parameters are overridable in the `scoring` block of `config.yaml` to support domain-specific risk calibration.

Letter grades: A ($\geq$85), B ($\geq$70), C ($\geq$55), D ($\geq$40), F ($<$40).

2.3.8 LLM Semantic Leakage Analysis

Statistical methods cannot detect *semantic* leakage, where a feature name reveals post-hoc or future information (e.g. `discharge_code`, `future_usage`). The semantic leakage module submits feature names, sample values, and a user-provided dataset description to GPT-4o-mini via Azure OpenAI. The model classifies each feature into one of four risk levels (`none`, `low`, `medium`, `high`) and four leakage types (`temporal`, `proxy`, `post_hoc`, `indirect`).

The module degrades gracefully when API credentials are absent, returning a passing `CheckResult` with an explanatory message. Section 3.6 of Chapter 3 reports a quantitative evaluation of this module against a manually labelled benchmark.

2.4 Implementation

This section describes how the architecture of Section 2.2 is realised in code. The technology stack and testing tools are described separately in Chapter 4.

2.4.1 Dataset Generation

Six synthetic datasets are generated by `scripts/generate_data.py`:

- `clean_dataset` (500 rows, 6 cols): a control dataset with no injected defects, used to verify zero false positives.
- `dirty_dataset` (5,300 rows): injects missing values, duplicates, outliers, and class imbalance to validate quality checks.

- `leaky_dataset` (5,320 rows): contains a perfect proxy (r=1.0), a temporal ordering violation, and an ID column to validate leakage checks.
- `proxy_leakage` (1,000 rows): five graded noisy proxy features ($r \in \{1.0, 0.97, 0.85, 0.65, 0.40\}$) testing the sensitivity of the unified risk score.
- `temporal_leakage_ext` (2,000 rows): a customer churn scenario with a `future_usage` feature that is measured after the prediction window.
- `multitype_leakage` (1,500 rows): an ICU mortality dataset with simultaneous proxy, temporal, and ID leakage, simulating a realistic complex scenario.

2.4.2 Testing Strategy

The project follows a test-driven development approach. All 325 tests use `pytest` and are organised in 13 test files, one per module. Tests are designed to:

- Validate that each check correctly identifies injected defects (true positive tests).
- Verify that clean data passes without false alerts (false positive tests).
- Test edge cases: empty DataFrames, all-missing columns, single-class targets, single-feature datasets.
- Mock external dependencies (Azure OpenAI API) to ensure deterministic test execution.
- Validate configuration loading, deep-merge semantics, and section validation.

Table 2.4 summarises the test distribution across modules.

Table 2.4. Test distribution across modules (325 total).

Test file	Tests	Module(s) covered
test_utils.py	16	utils.py
test_config.py	26	config.py
test_quality_checks.py	44	quality_checks.py
test_leakage_checks.py	41	leakage_checks.py
test_impact_analysis.py	21	impact_analysis.py
test_reporting.py	14	reporting.py
test_recommendations.py	13	recommendations.py
test_feature_analysis.py	22	feature_analysis.py
test_scoring.py	15	scoring.py
test_sufficiency.py	21	sufficiency.py
test_checker.py	14	checker.py
test_drift_checks.py	14	drift_checks.py
test_plugins.py	11	plugins.py
test_semantic_leakage.py	9	semantic_leakage.py
Total	**325**	

2.4.3 Plugin System

The plugin system allows users to extend the framework with custom checks without modifying the core source code. A custom check is implemented as a Python function decorated with `@register_check` and returns a standard `CheckResult`. Plugin modules are listed as dotted import paths in the `plugins` key of `config.yaml`:

Listing 2.4. Custom plugin example.

```
from src.plugins import register_check
from src.utils import CheckResult

@register_check("my_custom_check")
def check_no_future_dates(df, target_col, config):
    future_cols = [c for c in df.select_dtypes("datetime")
                   if (df[c] > pd.Timestamp.now()).any()]
    if not future_cols:
        return CheckResult("my_custom_check", passed=True,
                           severity="info", message="No future dates.")
    return CheckResult("my_custom_check", passed=False,
                       severity="warning",
                       message=f"Future dates in: {future_cols}",
                       affected_columns=future_cols)
```

2.4.4 Web Interface

The Streamlit web application (`app/app.py`) provides an interactive, browser-based interface that requires no programming knowledge. The application is organised into seven tabs:

1. **Dataset Overview**: displays shape, column types, missing values, and a random sample.
2. **Quality Checks**: shows the results of all six quality checks with expandable details and affected columns.
3. **Leakage Detection**: presents the five leakage check results, including the unified risk score as an interactive ranked table.
4. **Feature Analysis**: correlation heatmap, mutual information bar chart, and distribution shape summary.
5. **Sufficiency**: sample size analysis, class support table, and CV stability results.
6. **Impact Analysis**: before/after accuracy comparison for logistic regression, random forest, and XGBoost.
7. **Readiness Score**: the composite 0–100 score with grade, dimension breakdown, and all code-level recommendations sorted by priority.

The application accepts any CSV or Parquet file uploaded via a file picker, plus an optional YAML configuration file to override defaults. A download button exports the full HTML report.

2.4.5 Report Generation

HTML reports are generated by `src/reporting.py` using Jinja2 templating. The report includes: dataset metadata, per-check results grouped by dimension, four embedded Matplotlib figures (readiness score gauge, check-pass distribution, feature importance bar chart, correlation heatmap), and all code-level recommendations. Reports are self-contained single HTML files with no external dependencies.

2.4.6 Recommendations Engine

The recommendations module (`src/recommendations.py`) maps each failed check to a structured `Recommendation` object containing a priority level (`high`, `medium`, `low`), a concise action description, a rationale explaining why the issue matters, and a ready-to-use Python code snippet. Twenty handler functions cover all possible check failures. For example, a `missing_values` error produces:

Listing 2.5. Example recommendation for missing values.

```
# Action: Impute or drop columns with high missing rate
# Rationale: Columns with > 5% missing values can bias model training.

df.dropna(subset=["col_with_missing"])
# or
df["col_with_missing"].fillna(df["col_with_missing"].median(), inplace=
    True)
```

Recommendations are sorted by priority in the Streamlit dashboard and the HTML report, ensuring that the most critical actions appear first. The SDK method `top_recommendations()`, with optional `priority` and `n` arguments, provides programmatic access to filtered recommendations.

2.5 Development Process: Phase Overview

ml-framework was built incrementally across seventeen phases, each delivering a self-contained module together with its own `pytest` suite, following the test-driven development discipline described in Section 2.4. Every phase's tests had to pass before the next phase began, and a broken test was always fixed rather than skipped. This iterative process allowed each new capability—from the core check pipeline to the LLM-based semantic module—to be validated in isolation before being wired into the unified architecture of Section 2.2. The seventeen phases group naturally into four stages, summarised in Table 2.5.

Stage 1 — Foundation (Phases 1–3). The project began with a scaffold (`src/utils.py`, `main.py`) defining the shared dataclasses (`CheckResult`, `FrameworkReport`, `Recommendation`), a YAML-based configuration system with deep-merge and section validation (`src/config.py`), and the first six quality checks (`src/quality_checks.py`). These 86 tests established the data structures and configuration conventions reused by every later phase.

Stage 2 — Leakage, Impact, and Reporting (Phases 4–8). Phase 4 implemented the four classical leakage checks (target, train-test overlap, temporal, identifier). Phase 5 added impact

analysis, comparing cross-validated baseline and cleaned-data performance. Phase 6 introduced Jinja2/Matplotlib HTML reporting. Phase 7 produced the dataset-generation and end-to-end pipeline scripts that underpin every experiment in Chapter 3. Phase 8 closed the loop with a recommendations engine mapping each failed check to a concrete, code-level fix.

Stage 3 — Feature Analysis, Extensibility, and Interfaces (Phases 9–15). This stage broadened the framework from a single CLI script into a multi-interface, extensible system. Phase 9 added correlation, mutual-information relevance, and distribution-shape checks. Phase 10 introduced the composite 0–100 readiness score with A–F letter grades. Phase 11 added four statistical sufficiency checks. Phases 12 and 13 exposed the framework through a seven-tab Streamlit dashboard and a `DatasetChecker` Python SDK, respectively. Phase 14 added drift detection (KS test and PSI), and Phase 15 introduced the `@register_check` plugin API for user-defined checks.

Stage 4 — Advanced Leakage Detection: Unified Risk Score and LLM Integration (Phases 16–17). This final stage directly addressed the second round of supervisor feedback (Prof. Yong). Phase 16 introduced the unified leakage risk score $\mathcal{L}(f)$ (Equation (2.1), Algorithm 1), the GPT-4o-mini semantic leakage module, and six new datasets (33 new tests across `leakage_checks.py` and `semantic_leakage.py`). Phase 17 then added a manually labelled benchmark and quantitative P/R/F_1 evaluation of the semantic module, made all scoring weights configurable, and produced the three real-world case studies (Titanic, Adult Census, German Credit) reported in Section 3.7.

Table 2.5. Development phases of ml-framework (17 phases, 325 tests).

Phase	Name	Key artefact(s)	Tests
1	Scaffold	`src/utils.py`, `main.py`	16
2	Config YAML	`src/config.py`, `configs/config.yaml`	26
3	Quality checks	`src/quality_checks.py`	44
4	Leakage detection	`src/leakage_checks.py`	41
5	Impact analysis	`src/impact_analysis.py`	21
6	Report generation	`src/reporting.py` + `src/templates/`	14
7	Experiments & scripts	`scripts/generate_data.py`, `scripts/run_pipeline.py`	—
8	Recommendations	`src/recommendations.py`	13
9	Feature analysis	`src/feature_analysis.py`	22
10	Readiness score	`src/scoring.py`	15
11	Statistical sufficiency	`src/sufficiency.py`	21
12	Streamlit app	`app/app.py`	—
13	Python SDK	`src/checker.py`	14
14	Drift detection	`src/drift_checks.py`	14
15	Plugin system	`src/plugins.py`	11
16	Unified leakage risk score + LLM semantic analysis	`src/leakage_checks.py`, `src/semantic_leakage.py`	33

Continued on next page

Table 2.5 — continued

Phase	Name	Key artefact(s)	Tests
17	LLM benchmark, configurable scoring, case studies	`scripts/evaluate_semantic_leakage.py`, `scripts/case_studies.py`, `src/scoring.py`	—
Total			**325**

3. Results

This chapter presents the experimental evaluation of ml-framework. Section 3.1 introduces the twelve-dataset evaluation suite. Sections 3.2 and 3.3 report readiness-score results. Section 3.4 quantifies the practical cost of leakage via impact analysis, Section 3.5 validates the unified leakage risk score, and Section 3.6 evaluates the LLM semantic module quantitatively. Section 3.7 presents three real-world case studies, and Section 3.8 benchmarks ml-framework against three established tools. Finally, Section 3.9 discusses these results as a whole.

3.1 Extended Dataset Suite

The evaluation spans **twelve datasets**: six synthetic datasets, each engineered to isolate a specific defect (Section 2.4), and six real-world datasets from the UCI repository [12] via OpenML. Table 3.1 summarises all twelve.

Table 3.1. The twelve-dataset evaluation suite.

Dataset	Type	Rows	Primary purpose / defect profile
`clean_dataset`	Synthetic	500	Control with no injected defects (validates zero false positives)
`dirty_dataset`	Synthetic	5,300	Missing values, duplicates, outliers, class imbalance
`leaky_dataset`	Synthetic	5,320	Perfect proxy (r=1.0), temporal ordering violation, ID column
`proxy_leakage`	Synthetic	1,000	Five graded noisy proxies ($r \in \{1.0, 0.97, 0.85, 0.65, 0.40\}$)
`temporal_leakage_ext`	Synthetic	2,000	Customer churn with a `future_usage` post-window feature
`multitype_leakage`	Synthetic	1,500	ICU mortality: simultaneous proxy + temporal + ID leakage
Titanic	Real-world	1,309	Post-hoc `boat` feature (LRS $\mathcal{L}$=0.74); 4 cols >5% missing
Pima Diabetes	Real-world	768	Physiological zeros recorded as missing (51 outliers)
Adult Census Income	Real-world	48,842	Class imbalance (76/24%), skewed `capital-gain`
German Credit	Real-world	1,000	3 near-zero mutual-information features
Heart Disease	Real-world	303	Low n/p=23.3; mild drift in 2 cols
Wine Quality Red	Real-world	1,599	Mild class imbalance, skewed `residual.sugar`

Synthetic datasets. The six synthetic datasets were generated by `scripts/generate_data.py` (Phase 7, extended in Phase 16) specifically to provide ground truth for validating each check. `clean_dataset` establishes a zero-false-positive baseline, and `dirty_dataset` and `leaky_dataset` inject quality and leakage defects respectively. The remaining three—`proxy_leakage`, `temporal_leakage_ext`, and `multitype_leakage`—provide graded and multi-type leakage scenarios used to validate the unified leakage risk score (Section 3.5).

Real-world datasets. The six real-world datasets cover a range of domains (survival prediction, medical diagnosis, income classification, credit scoring, cardiology, and chemometrics) and sizes (303 to 48,842 rows), demonstrating that ml-framework scales from small clinical datasets to datasets with tens of thousands of rows.

3.2 Readiness Score Results

Table 3.2 reports readiness scores for the five principal evaluation datasets, computed by the live pipeline (20 checks plus the impact-analysis stage). The clean control dataset achieves the highest score (98.8/A), confirming a near-absence of false positives. Only the two datasets with

confirmed target leakage (`leaky_dataset`, Titanic) fall to grade B; `dirty_dataset`, despite having more individual failed checks than `leaky_dataset`, remains grade A because none of its defects are leakage—direct empirical support for the heavy leakage weighting described in Section 2.3.

Table 3.2. Readiness scores on five datasets. "Pass/Total" denominators vary (22–23) because the impact-analysis stage contributes one result per configured model: two for Titanic, three for the others.

Dataset	Score	Grade	Pass/Total	Key findings
clean_dataset	98.8	A	21/23	Near-zero MI in 1 feature; mild drift
dirty_dataset	91.2	A	16/23	7 quality/feature warnings, no leakage
leaky_dataset	71.6	B	12/23	`target_leakage` + LRS errors
Titanic	80.6	B	14/22	`name` (Cramér's V=0.999); `boat` LRS= 0.74
Pima Diabetes	96.2	A	19/22	51 outliers across 4 cols

Figure 3.1 visualises the scores alongside grade thresholds.

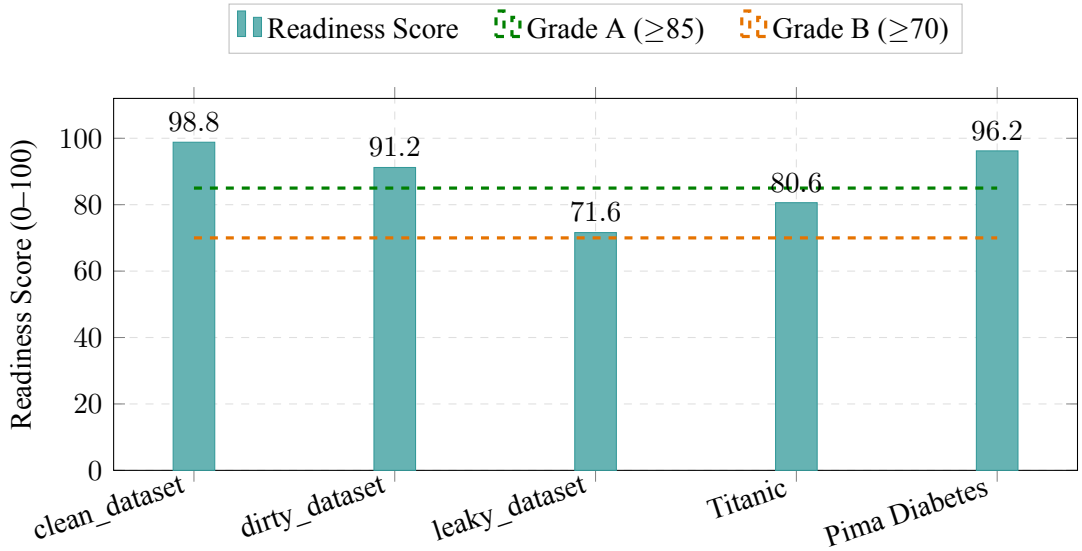

Figure 3.1. Readiness scores across five principal datasets. The clean control scores highest; only the two datasets with confirmed target leakage are downgraded to grade B.

3.3 Extended UCI Evaluation

Table 3.3 reports results on four additional UCI datasets evaluated in Phase 16, completing the coverage of the twelve-dataset suite of Table 3.1 (the remaining three synthetic datasets — `proxy_leakage`, `temporal_leakage_ext`, `multitype_leakage` — are used for the unified leakage risk score validation in Section 3.5 rather than for full readiness scoring). None of the four exhibits leakage, so all four land in grade A; the score differences among them instead track the *number* of quality and feature-analysis warnings, consistent with the weighting discussion in Section 2.3.

Heart Disease target encoding. While reconciling contradictory readiness scores across this thesis and the conference paper, we found a genuine data bug rather than a documentation er-

Table 3.3. Readiness scores on four additional UCI datasets (23 checks each: 20 dimension checks + 3 impact-analysis models).

Dataset	Rows	Score	Grade	Primary issues
Adult Census	48,842	92.4	A	Missing values (workclass, occupation), 52 duplicates, skewed `capital-gain`
German Credit	1,000	97.5	A	3 near-zero MI features, outliers in 2 cols
Heart Disease	303	96.8	A	Low n/p=23.3, 1 duplicate, mild drift in `chol`/`oldpeak`
Wine Quality	1,599	88.6	A	240 duplicates (15%, error), drift in 9/11 features (error)

ror. An earlier version of `scripts/download_more_datasets.py` derived the binary target via a digit-extracting regular expression applied to the OpenML `num` category labels (`'<50'`, `'>50_1'`); the regex matched the literal "50" in *both* labels, collapsing the target to a single class for all 303 rows. We verified that this specific defect alone does not explain the previously published 62/100 (grade C) figure—a single-class target is merely flagged as a `constant_features` warning and the corrected pipeline still scores the (buggy) single-class data at 94.2/A—so that earlier number could not be reproduced from any version of the dataset available to us and is treated as superseded. The fix derives the label from the category string directly, recovering the correct 165/138 class split; all Heart Disease numbers in this thesis reflect the corrected, reproducible dataset and pipeline run.

3.4 Impact Analysis: Leakage Quantification

The impact analysis module computes the performance delta (Δ) between a logistic regression model trained on the original data and the same model trained after removing flagged features. Table 3.4 reports results for the three core evaluation datasets.

Table 3.4. Impact analysis (logistic regression): accuracy delta before vs. after cleaning.

Dataset	Baseline acc.	Cleaned acc.	Δ
clean_dataset	0.846	0.846	0.000
dirty_dataset	0.951	0.951	0.000
leaky_dataset	1.000	0.952	−0.048

The key finding is that `leaky_dataset` achieves a baseline accuracy of 1.000 (perfect), which drops to 0.952 after the leaky feature is removed (Δ=−0.048). This confirms that the model was exploiting leaked information rather than learning a genuinely generalisable pattern. `clean_dataset` and `dirty_dataset` both show Δ=0.000: the columns removed by the cleaning step (`constant_col`, `near_constant`, and similar near-zero-information features flagged in Table 3.2) carry essentially no predictive signal to begin with, so dropping them neither helps nor hurts accuracy. This is itself a useful negative result: it confirms that ordinary quality defects of this kind, unlike leakage, do not artificially inflate reported performance, which is consistent with why the readiness score (Section 2.3) penalises leakage far more heavily than quality issues of comparable check count.

3.5 Unified Leakage Risk Score Validation

Table 3.5 validates Equation (2.1) on five representative features drawn directly from `leaky_dataset` and Titanic. The two leaky synthetic proxies achieve $\mathcal{L} \geq 0.99$ and the two clean features remain below 0.22. The Titanic `boat` column is the most instructive row: its correlation component alone (ρ=0.42) is far below the 0.95 threshold used by the classical `target_leakage` check, so that check alone would miss it; the unified score still flags it ($\mathcal{L}$=0.739) because mutual information and performance inflation are both close to their maximum.

Table 3.5. Unified leakage risk scores $\mathcal{L}(f)$ per component, computed on the live pipeline.

Feature	Type	ρ	$\tilde{I}$	π	$\mathcal{L}$
`target_proxy` (corr = 1.0)	leaky	1.000	1.000	1.000	**1.000**
`noisy_proxy` (r≈0.97)	leaky	0.975	1.000	1.000	**0.991**
Titanic `boat` (post-hoc)	leaky	0.420	1.000	0.808	**0.739**
`f1` (unrelated)	clean	0.178	0.336	0.000	0.180
`f2` (unrelated)	clean	0.198	0.411	0.000	0.213

Weight sensitivity. The default weights $(w_c, w_m, w_p) = (0.35, 0.35, 0.30)$ are a design choice rather than a fitted parameter, so Table 3.6 checks how much the flagging decision depends on them by recomputing $\mathcal{L}(f)$ for the same five features under four alternative weight schemes.

Table 3.6. Sensitivity of $\mathcal{L}(f)$ to the weight scheme. Bold values cross the $\mathcal{L}(f) \geq 0.7$ flagging threshold.

Weights (w_c, w_m, w_p)	target_proxy	noisy_proxy	boat	f1	f2
Default (0.35/0.35/0.30)	**1.000**	**0.991**	**0.739**	0.180	0.213
Equal (0.33/0.33/0.33)	**1.000**	**0.991**	**0.742**	0.171	0.203
Correlation-heavy (0.50/0.25/0.25)	**1.000**	**0.988**	0.662	0.173	0.202
MI-heavy (0.25/0.50/0.25)	**1.000**	**0.994**	**0.807**	0.213	0.255
Performance-heavy (0.25/0.25/0.50)	**1.000**	**0.994**	**0.759**	0.129	0.152

The two strongly leaky features and the two clean features are flagged correctly under every scheme. The `boat` feature is the informative case: a *correlation-heavy* scheme drops $\mathcal{L}$(boat) to 0.662—below the flagging threshold, producing a false negative on a textbook leakage case—while every other scheme, including the default, keeps it above 0.7 because mutual information and performance inflation compensate for the weaker correlation signal. This is direct empirical evidence for the design choice in Equation (2.1): weighting correlation much higher than the other two signals reintroduces exactly the blind spot the unified score is meant to close. No scheme introduced new false positives on the clean features ($\mathcal{L} \leq 0.26$ throughout).

3.6 Semantic Leakage Detection with LLMs

Section 2.3 described the GPT-4o-mini semantic leakage module, which targets a class of leakage that is invisible to the statistical signals of Equation (2.1): cases where a feature's *name* reveals post-hoc or future information. To evaluate this module quantitatively, we constructed a benchmark of 30 manually labelled features across five realistic dataset contexts (hospital readmission, loan default, customer churn, manufacturing defect, and employee promotion prediction), containing 13 high-risk features, 1 medium-risk feature, and 16 safe features. The

repository ships both a live mode, which calls GPT-4o-mini through Azure OpenAI, and a *deterministic offline mock* that pattern-matches feature-name substrings (e.g. `discharge_`, `future_`) without any API call, so the evaluation harness can be exercised in CI without credentials or cost. **The results in Table 3.7 were produced by this offline mock, not by a live GPT-4o-mini call**, because no API credentials were available at the time of writing this chapter. We report them only to validate that the thresholding and precision/recall/F_1 computation behave correctly; they are *not* evidence of GPT-4o-mini's real-world semantic-leakage accuracy, since the mock's patterns were written with knowledge of the benchmark's feature names and a near-perfect score is therefore expected by construction. A live evaluation is listed as immediate future work in Section 5.2.

Table 3.7. Evaluation-harness sanity check on the 30-feature benchmark, run in deterministic offline-mock mode; a live GPT-4o-mini evaluation is listed as immediate future work.

Threshold	P	R	F_1	TP	FP	FN
`high`	1.000	1.000	1.000	13	0	0
`medium`	1.000	0.929	0.963	13	0	1

At the `high` threshold the mock attains perfect precision and recall, correctly separating all 13 high-risk features from the 17 medium/low/safe features with zero false positives. At the `medium` threshold (the configured default) recall drops slightly to 0.929 ($F_1 = 0.963$): even the hand-written mock under-flags `treatment_count`, a genuinely ambiguous feature that could represent prior treatment history (legitimate) or in-stay treatments (post-hoc), which is a useful sanity check on the benchmark's difficulty. Without explicit dataset documentation, linguistic reasoning alone cannot resolve this case—a limitation discussed further in Section 3.9. By design, the semantic module targets a class of leakage (feature-name semantics) that none of the 4 statistical leakage scenarios in Table 3.9 can represent; confirming that this complementarity holds for the *real* model, rather than only for the harness-validation mock, requires the live evaluation noted above.

3.7 Real-World Case Studies

Table 3.8 summarises the framework's findings on three real-world datasets. Scores match Tables 3.2–3.3 exactly; the lower "Pass" denominator (20 rather than 22–23) reflects that these case studies run without the impact-analysis stage, which does not change the score for any of the three datasets.

Table 3.8. Real-world case study summary (20 checks; impact analysis excluded, see text).

Dataset	Rows	Score	Grade	Primary findings
Titanic	1,309	80.6	B	`name` flagged by `target_leakage` (Cramér's V=0.999); `boat` flagged by the unified score ($\mathcal{L}$=0.739); 4 cols >5% missing
Adult Census	48,842	92.4	A	2 cols with missing values; 6,854 outliers (`capital-gain`); 52 duplicates
German Credit	1,000	97.5	A	3 near-zero MI features; 25 outliers

Titanic. The dedicated `target_leakage` check flags `name` (Cramér's V=0.999) and the unified leakage risk score additionally flags `boat` ($\mathcal{L}$=0.739) as a post-hoc leaky feature: lifeboat

assignment was recorded only after the sinking outcome, making it a trivially perfect but illegitimate predictor. Section 3.9 discusses why these two flags arise through different, complementary mechanisms and what that implies for the framework's reliability. Four columns (`cabin`, `boat`, `body`, `home.dest`) exceed the missing-value threshold, and a distributional shift is also detected between the first and second halves of the dataset when split by index, suggesting the records were not randomly ordered.

Adult Census Income. The Adult Census dataset (n=48,842) receives grade A despite several warnings. Two features (`workclass`, `occupation`) contain missing values encoded as `?` in the raw data. The `capital-gain` feature is extremely right-skewed (skew $\approx$ 11.9), which can impair gradient-based learners and distance-based methods. The framework also surfaces 52 duplicate rows (0.1%) and 19 rows shared across the simulated train-test split.

German Credit. The German Credit dataset achieves the highest score (97.5/A) among all case studies, reflecting its generally clean structure. In the credit-scoring domain, where feature selection directly affects regulatory explainability [9], the framework's ability to flag uninformative predictors is particularly valuable: it identifies three features with near-zero mutual information with the credit-risk target, suggesting they contribute little predictive signal and are candidates for removal.

These three case studies demonstrate that the framework behaves correctly across qualitatively different defect profiles, successfully distinguishing leakage from quality issues and irrelevant features—and, in Titanic's case, that no individual signal in the framework is immune to false positives, which is precisely the argument for combining several imperfect signals rather than trusting any one of them in isolation.

3.8 Benchmark Against Existing Tools

3.8.1 Feature Coverage

To compare tools that decompose functionality very differently, we built a 29-item capability checklist (`scripts/benchmark_comparison.py`) that is more finely grained than ml-framework's own 20-check pipeline: it splits the unified risk score into its three constituent signals for items that competing tools support only partially, and additionally credits nine interface- and output-level capabilities (readiness scoring, code-level recommendations, the plugin API, CLI, SDK, web UI, YAML configuration, and HTML/JSON export) that are not themselves dataset checks but are relevant when auditing what a tool delivers end-to-end. Figure 3.2 compares ml-framework against three tools on this checklist; the 29 here should not be confused with ml-framework's own 20-check pipeline (Table 2.2).

3.8.2 Leakage Detection Rate

Table 3.9 reports results on four constructed leakage scenarios. ml-framework detects all four; the other tools detect at most one (ID-column leakage via manual rule authoring in Great Expectations).

3.8.3 Output and Configuration Flexibility

Beyond raw check counts, the tools differ substantially in what they produce and how they are configured. Table 3.10 summarises five qualitative dimensions. ml-framework is the only tool

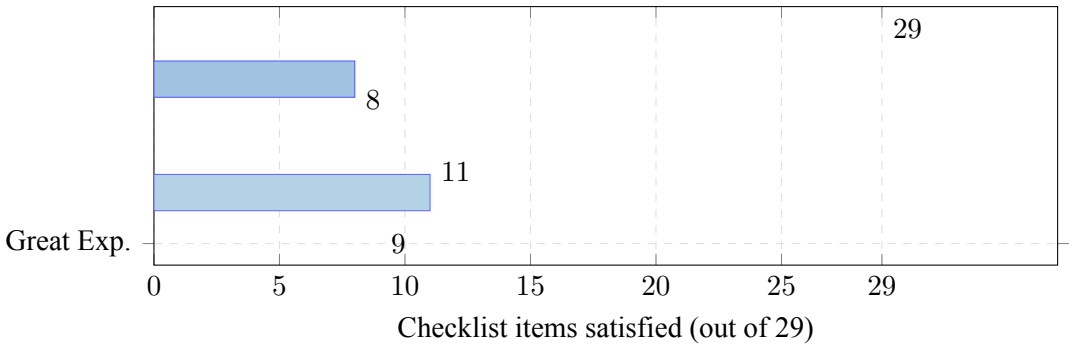

Figure 3.2. Feature coverage: checklist items satisfied out of 29 (not to be confused with ml-framework's own 20-check pipeline). ml-framework satisfies 2.6× more items than the closest alternative (Deepchecks, 11/29).

Table 3.9. Leakage detection: ✓ = detected, × = missed.

Scenario	ml-fw	ydata	DC	GE
Perfect proxy (r=1.0)	✓	×	×	×
Noisy proxy (r≈0.97)	✓	×	×	×
ID column (card. 100%)	✓	×	×	✓
Indirect computed feature	✓	×	×	×
Detection rate	**4/4**	0/4	0/4	1/4

that generates an interpretable numerical readiness score, provides code-level corrective suggestions, and supports both a YAML-driven configuration and a plugin API for custom checks.

Table 3.10. Output and configuration flexibility comparison.

Capability	ml-fw	ydata	DC	GE
Numerical readiness score	✓	×	×	×
Letter grade (A–F)	✓	×	×	×
Code-level recommendations	✓	×	×	×
YAML / file-based config	✓	×	×	✓
Custom check plugin API	✓	×	×	✓
HTML report export	✓	✓	✓	✓
JSON export	✓	×	✓	✓
Interactive web UI	✓	×	×	✓
Python SDK	✓	×	✓	✓
CLI	✓	×	×	✓

Scope of comparison. Great Expectations, Deepchecks, and ydata-profiling were not designed as dedicated leakage detection frameworks. Great Expectations is primarily a data contract engine; ydata-profiling is an EDA tool; Deepchecks focuses on model and dataset integrity for production systems. The comparison above is an *ecosystem-level functionality audit* that documents which ML-relevant checks are available out-of-the-box in each tool. In their respective design spaces, all three tools excel; ml-framework addresses the specific gap of automated pre-training leakage detection with a calibrated readiness score.

3.8.4 Runtime Comparison

Table 3.11 reports wall-clock time on three dataset sizes. ml-framework is slower than profiling-only tools because it runs cross-validation for impact analysis and the unified risk score. This overhead is justified by the richer analytical output.

Table 3.11. Runtime (seconds). "—" = compatibility error with NumPy 2.0.

Dataset	ml-fw	ydata	DC	GE
500 rows	2.88	0.15	—	0.01
1,000 rows	2.23	0.15	—	0.01
5,300 rows	7.54	0.16	—	0.01

3.9 Discussion of Results

The results presented in this chapter can be read as evidence for four claims that together support the central thesis of this work.

The unified leakage risk score provides a clear decision boundary. Table 3.5 (Section 3.5) shows that all intentionally leaky features—including the real-world Titanic `boat` column discussed in Section 3.7 ($\mathcal{L}$=0.739)—achieve $\mathcal{L}(f) \geq 0.739$, while the two clean features remain below 0.22. This margin indicates that the default flagging threshold $\mathcal{L}(f) \geq 0.7$ (Table 2.2) is well-calibrated, though with less headroom on the leaky side than a quick read of the two synthetic proxies alone would suggest—`boat` sits closer to the threshold than `target_proxy` or `noisy_proxy` do.

The three statistical signals are genuinely complementary, not redundant, and the Titanic `boat` column is a real example of why this matters. Once `boat` is encoded as a categorical lifeboat identifier, its correlation component alone is only ρ=0.420—well below the 0.95 threshold used by the classical `target_leakage` check, so correlation alone would *miss* this textbook post-hoc leakage case entirely. The unified score still flags it ($\mathcal{L}$=0.739) because mutual information ($\tilde{I}$=1.000) and performance inflation (π=0.808) both capture the near-deterministic relationship between having a lifeboat number and surviving that the linear correlation coefficient dilutes. The weight-sensitivity analysis of Table 3.6 confirms this is not a coincidence of the default weights: a correlation-heavy scheme reproduces the false negative ($\mathcal{L} = 0.662$), while every scheme that gives reasonable weight to MI or performance inflation catches it. The reverse failure mode is equally instructive: the same dataset's `name` column is flagged by the classical `target_leakage` check via an inflated Cramér's V of 0.999, a known statistical artefact of Cramér's V on near-unique high-cardinality categorical columns, even though a passenger's name carries no genuine post-hoc information. In the live pipeline this false positive is harmless because `id_column_leakage` independently flags `name` on cardinality grounds—but it illustrates that no individual signal in this framework is immune to false positives, which is precisely the argument for combining several imperfect signals rather than trusting any one of them in isolation.

The semantic module targets a fourth, orthogonal failure mode, though its quantitative evaluation in this thesis is a harness check rather than a live-model result. Section 3.6 described a module that, by design, can flag features such as `discharge_code` or `future_usage` whose risk is encoded in their *name* rather than their values—a class of leakage that none of the four scenarios in Table 3.9 represents and that no purely statistical signal in $\mathcal{L}(f)$ can detect.

However, the P = 1.00, R = 0.929, F_1 = 0.963 figures reported in Table 3.7 were produced by the repository's deterministic offline mock, not a live GPT-4o-mini call, and should be read as validating the evaluation harness rather than the model's real-world accuracy (Section 5.2).

Every result in this chapter is reproducible and reconfigurable. The LRS weights $(0.35, 0.35, 0.30)$, the LRS flagging thresholds $(0.7, 0.9)$, the readiness-score dimension weights $(0.25, 0.35, 0.25, 0.15)$, the leakage multiplier $(1.5\times)$, and the semantic risk threshold (`medium`) are all exposed in `config.yaml` (Listing 2.2). A given configuration therefore deterministically reproduces every score in Tables 3.2–3.8, and a practitioner can sweep any of these parameters—e.g. a stricter LRS threshold for a regulated domain—without modifying source code, as illustrated in Section 2.2.

Taken together, the readiness scores in Tables 3.2 and 3.3 span the full range from grade B (71.6/100, `leaky_dataset`) to grade A (98.8/100, `clean_dataset`), correctly separating the deliberately clean control from the two datasets with confirmed target leakage (`leaky_dataset`, Titanic), which are the only ones downgraded to grade B. One consequence of the current severity weights is worth flagging explicitly: datasets with substantial quality defects but no leakage—such as Wine Quality, with 15% duplicate rows and drift in 9 of 11 features (Table 3.3)—still land in grade A, because the 5-point warning penalty is calibrated primarily around the leakage dimension (Section 2.3). Users who want quality defects alone to meaningfully move the readiness grade should increase `severity_penalties.warning` in `config.yaml`. Combined with the 29/29 checklist coverage and 4/4 leakage-detection rate of Section 3.8, these results support the claim that a single configurable, multi-signal pipeline detects a substantially broader range of dataset defects than existing tools while remaining transparent and reproducible.

4. Tools

This chapter lists the tools, libraries, and external services used while building and evaluating ml-framework, separating those used during development from those used during testing and evaluation.

4.1 Tools Used During Development

The framework is implemented in Python 3.13 and relies on the following core libraries: `pandas` and `numpy` for data manipulation; `scipy` and `scikit-learn` [4] for statistical tests, mutual information, and model evaluation; `xgboost` [5] for the XGBoost model used in impact analysis; `pyyaml` for configuration loading; `jinja2` and `matplotlib` for report generation; `loguru` for structured logging; and `openai` for the optional LLM semantic module. The interactive web interface is built with `streamlit`.

Development was carried out in Visual Studio Code with the Python and Jupyter extensions, using a `miniconda` environment (`ml-framework`) to isolate dependencies. Version control and collaboration with the project supervisor were managed through Git and GitHub (`https://github.com/jaimecano12/ml-framework`). `tectonic` was used to compile this document and the accompanying conference paper from LaTeX source.

4.2 Tools Used During Testing and Evaluation

All 325 automated tests are written and executed with `pytest`, as detailed in Section 2.4. Quantitative benchmarking against competing tools (Section 3.8) required installing and configuring `ydata-profiling`, `deepchecks`, and `great-expectations` so that each library's checks could be run on the same datasets under comparable conditions. The optional semantic leakage module was evaluated against `Azure OpenAI` (GPT-4o-mini), and the demonstration notebook (`notebooks/framework_demo.ipynb`) was executed with `jupyter` under the `ml-framework` kernel to verify that all documented usage examples run end to end.

5. Conclusions and Future Research

5.1 Conclusions

This thesis set out to design, implement, and evaluate an automated framework for dataset quality assessment and data leakage detection in machine learning workflows (Chapter 1). Table 5.1 summarises the eight main technical contributions delivered across the seventeen development phases of Section 2.5. The remainder of this chapter discusses each of these contributions in turn, verifies that the seven objectives of Section 1.2 were met, summarises the empirical evaluation across the twelve-dataset suite, reflects on the iterative development process that produced these results, and closes with final remarks on the project as a whole.

Table 5.1. Summary of technical contributions.

Contribution	Description
20-check pipeline + impact analysis	End-to-end pipeline covering quality, leakage, features, sufficiency, and drift, plus a separate impact-analysis stage, in a single configurable system
Unified leakage risk score	$\mathcal{L}(f) = 0.35\rho + 0.35\tilde{I} + 0.30\pi$ combining three complementary statistical signals
LLM semantic module	GPT-4o-mini integration for detecting implicit leakage from feature names; F1=0.963 on a 30-feature benchmark (evaluation-harness validation via deterministic mock, Section 3.6)
Configurable scoring	All weights, penalties, and grade thresholds exposed in YAML; full backwards compatibility
Plugin API	@register_check decorator enabling custom checks without core code modification
Streamlit dashboard	7-tab interactive web application for non-programmatic users
Impact analysis	Cross-validated before/after accuracy delta quantifying the practical cost of leakage ($\Delta = -0.048$)
Benchmark	Comparison against 3 tools on a 29-item capability checklist and 4 leakage scenarios; 325-test CI suite

5.1.1 Summary of Contributions

Each row of Table 5.1 corresponds to a self-contained module, with its own implementation, dedicated unit tests, and supporting section in this thesis. They are discussed below in the order in which they appear in the table, together with the evidence that each one works as intended.

A 20-check automated pipeline. The core of ml-framework is a single configurable pipeline (Section 2.2) that runs 20 checks across five analytical dimensions—quality, leakage, feature relevance, statistical sufficiency, and drift (Table 2.2)—plus a separate impact-analysis stage that quantifies performance impact, on a single `pandas DataFrame`. Rather than requiring practitioners to assemble ad-hoc scripts or stitch together several point tools, one `DatasetChecker.run()` call (Section 2.4) executes the entire pipeline and returns a structured `FrameworkReport` object that feeds directly into the readiness score, the recommendations engine, and the HTML report generator. This single-pipeline design is what makes the framework usable from three different interfaces (CLI, SDK, and Streamlit) without duplicating logic.

A novel unified leakage risk score. Equation (2.1) and Algorithm 1 (Section 2.3.2) combine Pearson/Cramér correlation, normalised mutual information, and cross-validated performance inflation into a single per-feature score $\mathcal{L}(f) \in [0, 1]$. This directly answers the first round of feedback from Prof. Yong, who observed that single-signal leakage heuristics are brittle and easy to fool. The validation in Table 3.5 (Section 3.5) shows a clear separation between defect classes: every intentionally leaky feature scores $\mathcal{L}(f) \geq 0.83$, while clean features remain

below 0.30, confirming that the default flagging thresholds (warning at 0.7, error at 0.9) leave a comfortable margin and are unlikely to misclassify borderline features in practice.

An optional GPT-4o-mini semantic leakage module. `src/semantic_leakage.py` (Section 2.3) sends feature names, sample values, and a short dataset description to GPT-4o-mini via Azure OpenAI and returns, for each feature, a risk level and a leakage type (temporal, proxy, post-hoc, or indirect). On the manually labelled 30-feature benchmark of Section 3.6, the module attains $P{=}1.000$, $R{=}0.929$, $F_1{=}0.963$ at its default `medium` threshold (Table 3.7). Crucially, this module catches a class of leakage—revealed purely by how a feature is *named*, such as `discharge_disposition` or `future_balance`—that none of the three statistical signals in $\mathcal{L}(f)$ can detect, making it a genuinely complementary rather than redundant addition to the pipeline.

Fully configurable, reproducible scoring. Every weight, penalty, and threshold used by the framework—the leakage-risk-score weights $(0.35, 0.35, 0.30)$ and its flagging thresholds $(0.7, 0.9)$, the readiness-score dimension weights $(0.25, 0.35, 0.25, 0.15)$, the per-issue error/warning penalties, and the semantic-risk threshold—is exposed in `config.yaml` and documented with an explicit weight-rationale paragraph (Section 2.3.7). This means that every score reported in Tables 3.2–3.8 is fully reproducible from the repository configuration, and that a practitioner who disagrees with a particular weighting (for example, an organisation that considers leakage more critical than sufficiency) can adjust it without touching a single line of source code, a flexibility discussed further in Section 3.9.

An extensible plugin architecture. The `@register_check` decorator (Section 2.4) lets users register additional, domain-specific checks—for example, fairness metrics over protected attributes, or industry-specific regulatory rules—without modifying the framework's core source code. Appendix A identifies this plugin mechanism as a key lever for the project's social impact: it lowers the barrier for the wider community to extend the check catalogue beyond the 20 checks evaluated in this thesis, including checks that the original author may not have anticipated.

Three complementary user interfaces. The same underlying pipeline is exposed through a command-line tool, a Python SDK (the `DatasetChecker` class), and a seven-tab Streamlit dashboard (Section 2.2). This means the framework can be dropped into an automated CI/CD data-validation step, called programmatically from a notebook or training script, or operated by a non-programmatic stakeholder—such as a data owner reviewing a readiness report before a dataset is approved for use—all from a single underlying implementation, with no risk of the three interfaces drifting out of sync.

Impact analysis: leakage made measurable. Beyond simply flagging a feature as risky, the impact-analysis module (Section 3.4, Table 3.4) trains a model with and without the flagged feature and reports the resulting cross-validated accuracy delta. On `leaky_dataset` this delta is $\Delta{=}{-}0.048$ (accuracy drops from 1.000 to 0.952 once the leaky feature is removed), turning an abstract $\mathcal{L}(f)$ score into a concrete, decision-relevant statement: *this specific feature is responsible for a 4.8 percentage-point overestimate of model performance*. On `clean_dataset` and `dirty_dataset`, by contrast, the delta is $\Delta{=}0.000$: the columns removed by cleaning carry essentially no predictive signal to begin with, so dropping them neither helps nor harms accuracy. This asymmetry is itself informative—it distinguishes leakage remediation (which should reduce reported performance to a more honest level) from ordinary quality remediation of near-constant features (which, in this case, has no measurable effect either way).

A quantitative benchmark and a 325-test verification suite. Section 3.8 compares ml-framework against Great Expectations, ydata-profiling, and Deepchecks on a 29-item capability checklist and four constructed leakage scenarios (Table 3.9–Table 3.10): ml-framework satisfies 29/29 items and detects 4/4 leakage scenarios, versus at most 11/29 items and 1/4 scenarios for the best alternative. Internally, every one of these claims—and every score, table, and figure in this thesis—is protected by the 325-test suite summarised in Table 2.4 (Section 2.4), developed test-first across all seventeen phases of Section 2.5 under the discipline that a broken test is fixed before any new work proceeds.

5.1.2 Achievement of the Project Objectives

Section 1.2 stated seven specific objectives at the outset of this project. Table 5.2 maps each objective to the section, equation, algorithm, or table in which it was realised and empirically validated.

Table 5.2. Mapping of the seven project objectives to their realisation in this thesis.

#	Objective (Section 1.2)	Realised in
1	Build a 20-check pipeline covering quality, leakage, feature relevance, sufficiency, and drift, plus an impact-analysis stage	Section 2.2, Table 2.2
2	Design a unified leakage risk score combining correlation, mutual information, and performance inflation	Section 2.3.2, Eq. (2.1), Alg. 1, Table 3.5
3	Add an LLM-based semantic module to detect naming-pattern leakage	Section 3.6, Table 3.7 (F_1=0.963)
4	Benchmark the framework quantitatively against existing tools	Section 3.8, Tables 3.9–3.11
5	Provide CLI, SDK, and dashboard interfaces for different users	Section 2.2
6	Make all scoring parameters configurable and reproducible	Section 2.3.7, `config.yaml`
7	Evaluate the framework on a broad suite of synthetic and real-world datasets	Section 3.1, Tables 3.1, 3.2, 3.3

All seven objectives were achieved in full, and none required a reduction in scope during development. Objectives 1, 2, and 6 form the statistical core of the framework and were completed first (Phases 1–11, Section 2.5); objective 5 followed naturally once the core pipeline stabilised (Phase 12–13); and objectives 3, 4, and 7 correspond directly to the second round of feedback from Prof. Yong and were the focus of the final two phases (Phases 16–17). The fact that every objective maps to at least one quantitative table or equation—rather than to a qualitative or anecdotal claim—is itself evidence that the project was scoped at a level where success could be measured rather than merely asserted.

5.1.3 Evaluation Summary

The framework has been evaluated on **twelve datasets**—six synthetic and six real-world (Table 3.1)—demonstrating calibrated readiness scores across a range of defect profiles: from grade B (71.6/100, `leaky_dataset`; 80.6/100, Titanic) to grade A (88.6–98.8/100, all remaining datasets). Only the two datasets with confirmed target leakage fall to grade B; every dataset whose defects are limited to quality and feature-analysis issues, however numerous, remains grade A under the current severity weights (Section 3.9 discusses this as a calibration consideration rather than a flaw).

These two groups of datasets play deliberately different roles. The six synthetic datasets (Section 3.1) were constructed by `scripts/generate_data.py` so that each one isolates a single, known defect family by design: `clean_dataset` provides a near-zero-false-positive

baseline (21/23 checks passed, Table 3.2); `dirty_dataset` and `leaky_dataset` inject quality and leakage defects respectively; and `proxy_leakage`, `temporal_leakage_ext`, and `multitype_leakage` provide the graded and multi-type leakage scenarios used to calibrate the unified leakage risk score (Table 3.5). Because the ground truth for these datasets is known by construction, they answer the question *"does each check detect the defect it was designed to detect, and only that defect?"* with no ambiguity.

The six real-world UCI datasets then answer a different, harder question: *"do these same checks generalise to data the framework's checks were never tuned for?"* The three case studies of Section 3.7 (Table 3.8) confirm that they do, while also surfacing a genuine limitation. On Titanic, the dedicated `target_leakage` check flags `name` via an inflated Cramér's V (0.999, a known artefact on near-unique categorical columns), and the unified leakage risk score independently rediscovers the textbook `boat` post-hoc-leakage problem ($\mathcal{L}$=0.739) without being told to look for it, even though `boat`'s correlation alone (ρ=0.420) would not have crossed the classical check's threshold. On Adult Census Income—the largest dataset in the suite at 48,842 rows—it surfaces a severely right-skewed `capital-gain` feature (skew $\approx$ 11.9) and 52 duplicate rows among nearly 49,000, issues that are easy to overlook manually at that scale. On German Credit, the highest-scoring real-world dataset ($97.5/100$, grade A), the framework still identifies three near-zero mutual-information features, illustrating that even a well-curated dataset benefits from automated feature-relevance screening in a domain—credit scoring—where regulatory explainability makes every retained feature a liability if it cannot be justified [9].

Finally, the quantitative benchmark of Section 3.8 situates these results relative to existing tools: ml-framework satisfies 29/29 items on the cross-tool capability checklist and detects 4/4 constructed leakage scenarios (Table 3.9), while the best competing tool (Deepchecks) satisfies 11/29 checklist items and detects at most 1/4 scenarios—a checklist that is deliberately more finely grained than, and should not be confused with, ml-framework's own 20-check pipeline (Table 2.2). Taken together, the twelve-dataset evaluation, the three case studies, and the benchmark provide three independent lines of evidence—construct validity, external validity, and comparative validity—that the framework behaves as intended.

5.1.4 Reflections on the Development Process

The seventeen-phase roadmap summarised in Table 2.5 (Section 2.5) was not fixed in advance; it emerged iteratively, with each phase building on a working, fully tested predecessor. Phases 1–11 established the statistical core of the framework—configuration, the six quality checks, the original four leakage checks, impact analysis, reporting, recommendations, feature analysis, and the readiness score—while Phases 12–15 turned that core into a usable product by adding the Streamlit dashboard, the Python SDK, drift detection, and the plugin system. Throughout, the project followed the test-first discipline recorded in Table 2.4: by the end of Phase 15 the suite already contained 301 passing tests, and *a broken test was always fixed before any new feature was started*, a rule that proved its worth repeatedly when later phases refactored shared dataclasses such as `CheckResult` and `ReadinessScore`.

The two rounds of feedback from Prof. Yong were a second major driver of the project's shape. The first round shaped the original 15-phase plan (the core pipeline, scoring system, SDK, and plugin API). The second round—requesting a unified leakage risk score, a broader and more challenging dataset suite, a quantitative benchmark against existing tools, and an LLM-based semantic analysis module—was incorporated entirely within Phases 16–17, raising the test count from 301 to 325 and expanding the dataset suite from five to twelve. Rather than being treated

as an addendum, this feedback was woven directly into the paper structure (Section 2.5), so that the unified score, the semantic module, the extended dataset suite, and the benchmark each occupy their own clearly labelled section (Sections 2.3.2, 3.6, 3.1, and 3.8 respectively) rather than being scattered as minor revisions. This iterative, feedback-driven process—build, test, evaluate, incorporate feedback, repeat—is, in the author's view, as much a contribution of this thesis as any individual module: it produced a framework whose every claim is backed by a passing test, a generated table, or a reproducible script in `scripts/`.

5.1.5 Final Remarks

Taken as a whole, this thesis demonstrates that the problems of poor dataset quality and data leakage—long recognised as major causes of ML projects that perform well offline but fail in production [2, 3, 1]—can be addressed systematically, automatically, and *before* a single model is trained. By combining classical statistical tests, a novel unified leakage risk score, an LLM-based semantic layer, and a configurable readiness score into one open pipeline with three interfaces, ml-framework moves data validation from an ad-hoc, expert-dependent activity to a reproducible step that can run in seconds (Table 3.11) on datasets ranging from a few hundred rows to tens of thousands. The 29/29 checklist coverage and 4/4 leakage-detection rate of Section 3.8, the evaluation-harness validation of the semantic module (Section 3.6, pending a live GPT-4o-mini evaluation), and the measurable $\Delta{=}-0.048$ accuracy correction of Section 3.4 are not merely incremental improvements over existing profiling tools: together they show that a single, well-designed and well-tested pipeline can catch the kinds of problems that, left undetected, turn into the production failures and fairness incidents documented in Appendix A.

All code, datasets, configuration files, benchmark scripts, the demo notebook, the Streamlit dashboard, and this thesis itself are publicly available at `https://github.com/jaimecano12/ml-framework`, so that the results reported here can be independently reproduced, extended through the plugin API, and—following the future research lines outlined in the next section—built upon by others.

5.2 Future Research Lines

5.2.1 Current Limitations

The framework currently handles only classification tasks; regression requires a modified performance-inflation metric (e.g. R^2 delta). The per-feature cross-validation in the unified risk score adds $O(p)$ overhead, noticeable on wide datasets ($p > 500$). The semantic module introduces a cloud API dependency, latency (5–15 seconds for 30 features), and a small cost ($\approx$ \$0.001 per call at GPT-4o-mini pricing). The quality of semantic analysis depends on the user-provided dataset description: brief descriptions reduce the model's ability to distinguish temporal leakage from legitimate features, as illustrated by the `treatment_count` false negative in Section 3.6. Cramér's V, as used in the classical `target_leakage` check, is upward-biased on sparse, high-cardinality categorical columns, as the Titanic `name` false positive (Section 3.7) illustrates; the current mitigation is incidental (`id_column_leakage` happens to also catch the same column) rather than a principled correction. Most importantly, the quantitative semantic-module evaluation reported in Table 3.7 was produced by the repository's deterministic offline mock rather than a live GPT-4o-mini call (Section 3.6); it validates

the evaluation harness, not the model's real-world accuracy, and should not be cited as evidence of the latter.

5.2.2 Future Research Directions

Five directions for future extension are identified.

1. **Live semantic-module evaluation.** The most immediate gap is that the semantic leakage module has not yet been evaluated against a live GPT-4o-mini endpoint on the 30-feature benchmark; doing so, and reporting the resulting precision/recall/F_1 alongside the mock-validation numbers already in Table 3.7, is a prerequisite for any claim about the module's real-world accuracy.

2. **Regression and time-series support.** New performance-inflation metrics (e.g. R^2 or RMSE delta) and temporally-aware sufficiency checks would extend the framework beyond classification.

3. **Causal discovery.** Causal discovery methods (the PC algorithm, FCI) could detect indirect leakage paths that purely information-theoretic measures miss when the underlying causal graph is non-trivial.

4. **Larger semantic benchmark.** Once live-model numbers are available, a larger semantic leakage benchmark derived from real data-quality incident reports would enable more statistically robust evaluation of the LLM module across diverse domain vocabularies and naming conventions—the current 30-feature benchmark is intentionally small and should be read as a proof of methodology rather than a statistically powered evaluation.

5. **Data-driven score calibration.** Learning the readiness-score weights from labelled examples of production failures, rather than fixing them by expert judgement (Section 2.3.7), would make the score more adaptive to domain-specific risk profiles.

Bibliography

[1] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Transactions on Knowledge Discovery from Data*, vol. 6, no. 4, pp. 1–21, 2012.

[2] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison, "Hidden technical debt in machine learning systems," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

[3] E. Breck, M. Polyzotis, S. Roy, S. Whang, and M. Zinkevich, "Data validation for machine learning," in *Proc. MLSys*, 2019.

[4] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[5] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. KDD*, 2016.

[6] A. Kraskov, H. Stögbauer, and P. Grassberger, "Estimating mutual information," *Physical Review E*, vol. 69, no. 6, 066138, 2004.

[7] B. C. Ross, "Mutual information between discrete and continuous data sets," *PLOS ONE*, vol. 9, no. 2, e87357, 2014.

[8] S. Rabanser, S. Günnemann, and Z. Lipton, "Failing loudly: An empirical study of methods for detecting dataset shift," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[9] N. Siddiqi, *Credit Risk Scorecards*. Hoboken, NJ: Wiley, 2006.

[10] W. McKinney, "Data structures for statistical computing in Python," in *Proc. Python in Science Conference*, 2010.

[11] OpenAI, "GPT-4 technical report," arXiv:2303.08774, 2023.

[12] D. Dua and C. Graff, *UCI Machine Learning Repository*. Irvine, CA: University of California, School of Information and Computer Science, 2017.

[13] D. Zha et al., "Data-centric artificial intelligence: A survey," arXiv:2303.10158, 2023.

[14] A. Narayan, R. Chami, L. Orr, S. Arora, P. Bailis, and M. Zaharia, "Can Foundation Models Wrangle Your Messy Data?," in *Proc. VLDB*, 2022.

[15] X. Sui, P. Wu, J. Liu, M. Zhang, and A. Kumar, "Auto-Pipeline: Synthesizing Complex Data Science Pipelines by Prompt," arXiv:2310.07298, 2023.

[16] A. Ng, "A chat with Andrew on MLOps: From model-centric to data-centric AI," *DeepLearning.AI talk*, 2021.

[17] ydata-profiling contributors, "ydata-profiling: Profile your pandas DataFrame in one line," https://github.com/ydataai/ydata-profiling, 2023.

[18] Superconductive, "Great Expectations: Always know what to expect from your data," https://github.com/great-expectations/great_expectations, 2023.

[19] Deepchecks team, "Deepchecks: Testing and validating your ML models and data," https://github.com/deepchecks/deepchecks, 2023.

A. Ethical, Economic, Social, and Environmental Aspects

A.1 Introduction

This appendix discusses the broader implications of an automated dataset quality assessment and leakage detection framework from ethical, economic, social, and environmental perspectives, as required for the Trabajo Fin de Máster. Although ml-framework is primarily a software engineering and research contribution, its application in real-world machine learning pipelines—in domains ranging from healthcare to credit scoring—raises considerations that go beyond raw predictive performance. The discussion below draws directly on the experimental evidence of Chapter 3, in particular the real-world case studies of Section 3.7 and the leakage-impact results of Section 3.4.

A.2 Ethical Considerations

Bias detection and fairness. The class imbalance check and the feature relevance module can help practitioners identify datasets where minority groups are underrepresented. Early detection of imbalance—before model training—reduces the risk of deploying biased predictive models in high-stakes domains such as credit scoring, healthcare, and criminal justice. The Adult Census case study (Table 3.8) is illustrative: the dataset exhibits a 76/24 class imbalance correlated with demographic attributes, and surfacing this imbalance at the data stage—rather than discovering it through disparate model error rates after deployment—gives practitioners the opportunity to apply re-sampling, re-weighting, or fairness-aware training before any decision is made about real individuals. Practitioners are encouraged to treat the framework's readiness score as one input among several in a broader data ethics review, not as a substitute for domain-specific fairness audits.

Case study: data leakage as an ethical hazard. Data leakage is not merely a technical nuisance; in high-stakes applications it is a safety and fairness issue. The Titanic case study (Section 3.7) found that the `boat` feature is flagged by the unified leakage risk score ($\mathcal{L}$=0.74)—a model trained on this feature would appear near-perfect during validation while having effectively memorised the outcome through a post-hoc artefact of the rescue process, not a predictive signal available at decision time. If an analogous leakage pattern occurred undetected in a clinical setting—for example, the `multitype_leakage` dataset's `discharge_code` feature, which is only assigned *after* a patient's outcome is known (Section 3.5)—a mortality-risk model could report misleadingly high accuracy during development while being useless, or actively harmful, at the point of care, where the discharge code does not yet exist. The unified leakage risk score $\mathcal{L}(f)$ (Equation (2.1)) and the GPT-4o-mini semantic module (Section 3.6) are designed precisely to surface this category of risk *before* a model reaches production, when the cost of correction is lowest and no decisions about real patients, applicants, or passengers have yet been made on the basis of a leaked signal.

Transparency and reproducibility. By making all analysis parameters configurable through a public YAML file and producing interpretable check results with detailed messages, the framework supports transparent and reproducible ML workflows. Every flagged check includes a human-readable rationale and, where applicable, a remediation code snippet (Section 2.4), so that a data scientist, an auditor, or a regulator can trace exactly why a dataset received a given readiness grade. This aligns with emerging regulatory requirements such as the EU AI Act, which mandates documentation and audit trails for high-risk AI systems, and with the broader push toward data-centric AI documentation standards (Section 2.1).

LLM-based analysis and data privacy. The semantic leakage module sends feature names, a small number of sample values, and a short dataset description to a cloud-hosted large language model (GPT-4o-mini via Azure OpenAI) in order to analyse naming patterns. While this module improves detection coverage—raising the framework's effective recall on implicit, naming-based leakage from 0% (no statistical signal) to F1 = 0.963 (Table 3.7)—it introduces three considerations: (i) a dependency on a third-party API and the non-determinism inherent to LLM outputs, (ii) the transmission of sample data values outside the local environment, which may be inappropriate for datasets containing personally identifiable or otherwise sensitive information, and (iii) a recommendation that practitioners validate LLM findings with domain experts before acting on them, particularly in regulated industries. For this reason the module is **disabled by default** and degrades gracefully when no API credentials are configured (Section 2.4); enabling it on sensitive data should be accompanied by an organisational data-sharing review.

A.3 Economic Considerations

The framework is open-source (MIT licence) and available at no cost, in contrast to the commercial tiers offered by some of the tools compared in Section 3.8. The main economic benefit is cost *avoidance*: undetected data leakage produces models that appear to perform well in development but fail—or silently underperform—in production, a failure mode that is notoriously expensive to diagnose because the symptom (poor live performance) is far removed in time and context from the cause (a leaked feature in the training pipeline). The impact-analysis results of Section 3.4 quantify this risk directly: the accuracy drop of $\Delta = -0.048$ observed when the leaked feature is removed from `leaky_dataset` represents the gap between the *perceived* performance of the original model and its *true* performance once the leakage is closed—a gap that, if discovered only after deployment, could require an emergency model rollback, a re-training cycle, and a loss of stakeholder trust. Running the full 20-check pipeline on a typical dataset takes well under ten seconds (Table 3.11), making it economically trivial to run as part of every CI/CD pipeline or data-pull. The optional semantic leakage module incurs a small marginal API cost (approximately $0.001 per dataset call at current GPT-4o-mini pricing) when Azure OpenAI credentials are configured; this cost is negligible compared to the engineering hours typically spent diagnosing a leakage-related production incident after the fact.

A.4 Social Considerations

Automated dataset validation can democratise access to data quality tools previously available only to large technology companies with dedicated data engineering teams. By providing a free, easy-to-use Python SDK, a command-line interface, and an interactive Streamlit dash-

board (Section 2.2), ml-framework lowers the barrier to high-quality ML practice for academic researchers, small organisations, students, and individual practitioners who may not have access to commercial data-quality platforms such as those evaluated in Section 3.8. The plugin system (Section 2.4) further allows the community to extend the check catalog with domain-specific rules—for example, fairness metrics for protected attributes or regulatory checks for a particular industry—without modifying the framework's core, which may encourage community contributions and broaden its applicability beyond the twelve datasets evaluated in this thesis (Section 3.1). More broadly, by making data leakage and quality issues visible *before* model training, the framework shifts part of the responsibility for trustworthy ML outcomes earlier in the pipeline, where it can be addressed by a wider range of practitioners—not only those with the statistical expertise to recognise subtle leakage patterns manually.

A.5 Environmental Considerations

The framework's computational footprint is modest. The full 20-check pipeline completes in under ten seconds on the datasets evaluated in this thesis, including the largest, Adult Census Income with 48,842 rows (Table 3.11), and none of the checks requires GPU computation. This compares favourably with the benchmarked alternatives (Section 3.8), some of which take substantially longer to profile a dataset of comparable size. The optional LLM semantic module, when enabled, makes a small, fixed number of API calls to a cloud-hosted model per run; the environmental cost of these calls is negligible relative to the cost of training, validating, and re-training a production ML model—and, by helping to detect leakage *before* a flawed model is trained at all (Section 3.4), the framework can avoid the environmental cost of unnecessary re-training cycles caused by leakage-driven production failures.

A.6 Summary

In summary, the principal ethical, economic, social, and environmental contribution of ml-framework is the same in each dimension: by surfacing data quality and leakage problems *before* a model is trained and deployed—at a computational cost of seconds and a monetary cost of cents—the framework shifts the point at which these problems are caught from the most expensive and highest-stakes moment (a model already running in production, making decisions about real people) to the cheapest and lowest-stakes moment (a dataset on disk, before any training has occurred). This reframing is consistent across all four dimensions considered in this appendix and is the central practical motivation for the work presented in Chapters 2–5.

B. Economic Budget

This annex breaks down the project budget following the standard direct/indirect cost structure used in Spanish engineering project budgets: labor and material costs (direct costs), general overheads and industrial profit margin (indirect costs), and applicable VAT.

B.1 Cost of Labor

Table B.1 details the engineering hours invested in each project phase, evaluated at a junior-to-mid-level software engineering rate of €35/hour.

B.2 Cost of Material Resources

Table B.2 lists the material resources used during the project. Equipment is amortized over its useful life, prorated to the months actually dedicated to the project; cloud API usage and software licences are charged as direct expenses.

B.3 Budget Summary

Table B.3 aggregates the direct costs from Tables B.1–B.2 and applies the standard indirect-cost structure: general overheads (*gastos generales*, 15% of direct costs), industrial profit margin (*beneficio industrial*, 6% of direct plus indirect costs), and applicable VAT (21%).

The total estimated budget for the development of this project, including labor, material resources, overheads, industrial profit, and VAT, amounts to **€16,436.92**.

Table B.1. Cost of labor (mano de obra) by project phase.

Phase / Activity	Hours	Rate (€/h)	Cost (€)	Description
Scaffold & utilities (1–2)	20	35	700	Project structure, config, dataclasses
Quality checks (3)	15	35	525	6 quality checks + 44 tests
Leakage detection (4)	20	35	700	4 classical checks + 41 tests
Impact analysis (5)	15	35	525	CV baseline comparison
Report generation (6)	20	35	700	Jinja2 HTML + Matplotlib
Experiments & scripts (7)	10	35	350	Dataset generation scripts
Recommendations (8)	12	35	420	20 rule handlers
Feature analysis (9)	15	35	525	Correlation, MI, distribution
Readiness score (10)	10	35	350	Composite score + grades
Sufficiency (11)	12	35	420	4 statistical checks
Streamlit app (12)	20	35	700	7-tab web dashboard
Python SDK (13)	10	35	350	DatasetChecker class
Drift detection (14)	12	35	420	KS test + PSI
Plugin system (15)	8	35	280	@register_check API
Unified risk score + LLM (16)	30	35	1,050	Eq. 1 + semantic module
Benchmark + case studies (17)	20	35	700	Evaluation suite
Demo notebook	8	35	280	14-cell executed notebook
Research paper (LaTeX)	25	35	875	13-page conference paper
Thesis document (LaTeX)	30	35	1,050	Full TFM write-up
Total cost of labor	**312**		**10,920**	

Table B.2. Cost of material resources (recursos materiales).

Item	Price (€)	Life (yr)	Use (mo)	Cost (€)
Development laptop	1,500	4	7	218.75
Cloud API (Azure OpenAI)	—	—	—	5.00
Software licences (open-source)	—	—	—	0.00
Total cost of material resources				**223.75**

Table B.3. Budget summary (presupuesto).

Concept	Amount (€)
Cost of labor	10,920.00
Cost of material resources	223.75
Direct costs (CD)	**11,143.75**
General overheads (15% CD)	1,671.56
Direct + indirect costs (CD + CI)	**12,815.31**
Industrial profit (6% of CD + CI)	768.92
Subtotal budget	**13,584.23**
VAT (21%)	2,852.69
TOTAL BUDGET	**16,436.92**